

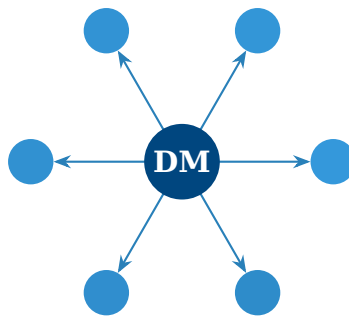
NTUA

Data Mining

Οδηγός Μελέτης για Εξετάσεις

Ακαδημαϊκό Έτος 2025-26

Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών



Θέματα που καλύπτονται:

- | | | |
|--------------------|-----------------------|----------------|
| ● ER Model | ● Relational Algebra | ● SQL |
| ● Query Processing | ● Data Preprocessing | ● Data Streams |
| ● HMMs/Sequences | ● OLAP/Data Warehouse | ● RAG/LLMs |

18 Ιανουαρίου 2026

Περιεχόμενα

1	ER Model & Database Design	1
1.1	Βασικές Έννοιες	1
1.1.1	Entity Types	1
1.1.2	Attribute Types	1
1.2	Cardinality & Participation	2
1.2.1	Participation Constraints	2
1.3	Solved Exam Problems	2
1.3.1	2024-25 Exam Style: ER to Relational	2
1.3.2	Cardinality Rules for Conversion	3
1.4	Common Mistakes	3
1.5	Quick Reference	4
2	Relational Model & Algebra	5
2.1	Βασικοί Τελεστές	5
2.1.1	Selection (σ)	5
2.1.2	Projection (π)	6
2.1.3	Set Operations ($\cup, \cap, -$)	6
2.2	Join Operations	6
2.2.1	Natural Join ($\bowtie$)	7
2.2.2	Theta Join ($\bowtie_{\theta}$)	7
2.2.3	Outer Joins	7
2.3	Solved Exam Problems	7
2.3.1	2024-25 Exam Question 6 Style	8
2.3.2	Division Operation	8
2.4	Operator Precedence	9
2.5	Common Mistakes	9
2.6	Quick Reference	10
3	SQL Basics	11
3.1	SELECT Basics	11
3.1.1	Column Selection	11
3.1.2	WHERE Clause	12
3.2	Aggregate Functions	12
3.3	GROUP BY & HAVING	13
3.4	Solved Exam Problems	13
3.4.1	2024-25 Exam Question 13 Style	13
3.4.2	2024-25 Exam Question 14 Style	14
3.4.3	Finding Maximum with Details	14
3.5	Common Patterns	14
3.6	Common Mistakes	15
3.7	Quick Reference	16

4	Advanced SQL	17
4.1	JOIN Types Visualization	17
4.2	JOIN Examples	17
4.3	Self-Join	18
4.4	Subqueries	18
4.4.1	Types of Subqueries	18
4.4.2	EXISTS vs IN	19
4.5	Window Functions	19
4.6	Common Table Expressions (CTE)	20
4.7	Solved Exam Problems	20
4.7.1	2024-25 Exam Question 7 Style: Join Algorithm	20
4.7.2	Complex Query with Multiple Joins	21
4.8	Common Mistakes	21
4.9	Quick Reference	22
5	Query Processing & Optimization	23
5.1	Query Evaluation Pipeline	23
5.2	Materialized vs Pipelined Evaluation	23
5.2.1	Materialized Evaluation	23
5.2.2	Pipelined Evaluation	24
5.3	Join Algorithms	24
5.3.1	Nested Loop Join	24
5.3.2	Hash Join	25
5.3.3	Merge Join (Sort-Merge Join)	25
5.3.4	Join Algorithm Comparison	25
5.4	Solved Exam Problems	26
5.4.1	2024-25 Exam Question 5 Style	26
5.4.2	Cost Estimation	26
5.5	Query Optimization Rules	27
5.6	Common Mistakes	27
5.7	Quick Reference	27
6	Data Preprocessing	29
6.1	Attribute Types (ΕΞΕΤΑΣΤΙΚΟ ΘΕΜΑ!)	29
6.1.1	NOIR Scale	29
6.2	Distance Measures	30
6.2.1	Euclidean Distance	30
6.2.2	Manhattan Distance	30
6.2.3	Cosine Similarity	31
6.2.4	Jaccard Similarity	31
6.3	Data Discretization	32
6.3.1	Equal-Width Partitioning	32
6.3.2	Equal-Frequency (Equal-Depth) Partitioning	32
6.4	Data Normalization	32
6.5	Missing Value Handling	33
6.6	Common Mistakes	33
6.7	Quick Reference	33
7	Data Streams & Bloom Filters	35
7.1	Stream Processing Model	35
7.2	Sampling Techniques	35
7.2.1	Reservoir Sampling	36

7.2.2 Sliding Window	36
7.3 Bloom Filters	36
7.3.1 Structure	36
7.3.2 Operations	37
7.3.3 False Positive Probability	37
7.4 Count-Min Sketch	38
7.5 Flajolet-Martin Algorithm	38
7.6 Solved Exam Problems	39
7.7 Common Mistakes	39
7.8 Quick Reference	40
8 Discrete Sequences & HMMs	41
8.1 Hidden Markov Models (HMMs)	41
8.1.1 Model Structure	41
8.1.2 Three Fundamental Problems	42
8.2 Viterbi Algorithm (EXAM QUESTION!)	42
8.3 Sequential Pattern Mining	43
8.3.1 Definitions	43
8.3.2 GSP Algorithm (Generalized Sequential Pattern)	44
8.4 Common Mistakes	44
8.5 Quick Reference	45
9 OLAP & Data Warehousing	47
9.1 Data Warehouse Concepts	47
9.1.1 OLTP vs OLAP	48
9.1.2 Data Cube Structure	48
9.2 Star vs Snowflake Schema	48
9.2.1 Star Schema	48
9.2.2 Snowflake Schema	49
9.3 OLAP Operations (EXAM QUESTIONS!)	49
9.3.1 Roll-up & Drill-down	49
9.3.2 Slice & Dice	49
9.3.3 Pivot (Rotate)	49
9.4 Solved Exam Problems	50
9.4.1 2024-25 Exam Questions 1-3 Style	50
9.4.2 2024-25 Exam Question 15 Style	50
9.5 Common Mistakes	51
9.6 Quick Reference	51
10 RAG & Retrieval-based LLMs	53
10.1 RAG Architecture	53
10.2 Retrieval Methods	53
10.2.1 Sparse Retrieval (Traditional)	54
10.2.2 Dense Retrieval (Neural)	54
10.3 Vector Databases	54
10.4 Chunking Strategies	55
10.5 RAG Evaluation	55
10.6 Common Challenges	56
10.7 Advanced RAG Techniques	56
10.8 Quick Reference	57
11 Formulas Cheatsheet	59

11.1 Relational Algebra	59
11.2 SQL Essentials	59
11.3 Association Rules	59
11.4 Distance Measures	60
11.5 Classification Metrics	60
11.6 Decision Trees	60
11.7 HMM - Viterbi Algorithm	60
11.8 Bloom Filter	60
11.9 OLAP Operations	61
11.10 Data Preprocessing	61
11.11 Query Processing	61
11.12 Quick Reference Table	62
12 Συλλογή Λυμένων Ασκήσεων	63
12.1 E-R Μοντέλο και Σχεδιασμός Βάσεων Δεδομένων	63
12.1.1 Άσκηση 1.1: Σχεδιασμός E-R από Απαιτήσεις	63
12.1.2 Άσκηση 1.2: Μετατροπή E-R σε Σχεσιακό (M:N)	64
12.1.3 Άσκηση 1.3: Weak Entity Μετατροπή	65
12.1.4 Άσκηση 1.4: Cardinality Constraints	66
12.1.5 Άσκηση 1.5: Σύνθετο Schema (Hospital System)	67
12.2 Σχεσιακή Άλγεβρα	68
12.2.1 Άσκηση 2.1: Selection και Projection	68
12.2.2 Άσκηση 2.2: Natural Join	69
12.2.3 Άσκηση 2.3: Theta Join	69
12.2.4 Άσκηση 2.4: Set Difference	70
12.2.5 Άσκηση 2.5: Intersection με Projection	70
12.2.6 Άσκηση 2.6: Division (Απλή)	71
12.2.7 Άσκηση 2.7: Division (Σύνθετη)	72
12.2.8 Άσκηση 2.8: Σύνθετη Έκφραση	72
12.3 SQL	73
12.3.1 Άσκηση 3.1: SELECT με WHERE	73
12.3.2 Άσκηση 3.2: JOINS (INNER, LEFT)	74
12.3.3 Άσκηση 3.3: GROUP BY με COUNT	74
12.3.4 Άσκηση 3.4: GROUP BY με HAVING	75
12.3.5 Άσκηση 3.5: Subquery με IN	75
12.3.6 Άσκηση 3.6: Correlated Subquery	76
12.3.7 Άσκηση 3.7: NOT EXISTS Pattern	76
12.3.8 Άσκηση 3.8: SQL Division (Double NOT EXISTS)	77
12.3.9 Άσκηση 3.9: Multiple JOINS με Aggregation	77
12.3.10 Άσκηση 3.10: Complex Query	78
12.4 Query Processing	78
12.4.1 Άσκηση 4.1: Pipelining vs Materialization	79
12.4.2 Άσκηση 4.2: Επιλογή Join Algorithm	79
12.4.3 Άσκηση 4.3: Query Optimization (Push Selection)	80
12.4.4 Άσκηση 4.4: Cost Comparison	81
12.5 Προεπεξεργασία Δεδομένων	82
12.5.1 Άσκηση 5.1: Ταξινόμηση Attribute Types (NOIR)	82
12.5.2 Άσκηση 5.2: Equal-Width Binning	83
12.5.3 Άσκηση 5.3: Equal-Frequency Binning	84
12.5.4 Άσκηση 5.4: Smoothing by Bin Means	84
12.5.5 Άσκηση 5.5: Smoothing by Bin Boundaries	85
12.5.6 Άσκηση 5.6: Min-Max Normalization	86

12.5.7Άσκηση 5.7: Z-Score Standardization	86
12.5.8Άσκηση 5.8: Υπολογισμός Αποστάσεων	87
12.6Data Streams	88
12.6.1Άσκηση 6.1: Bloom Filter Insert & Lookup	88
12.6.2Άσκηση 6.2: Bloom Filter False Positive	89
12.6.3Άσκηση 6.3: Count-Min Sketch Query	89
12.6.4Άσκηση 6.4: Count-Min Sketch Update	90
12.6.5Άσκηση 6.5: Error Bounds	91
12.6.6Άσκηση 6.6: Bloom Filter vs Count-Min Sketch	91
12.7HMM και Διακριτές Ακολουθίες	92
12.7.1Άσκηση 7.1: Υποακολουθίες	92
12.7.2Άσκηση 7.2: Forward Algorithm - Απλό	93
12.7.3Άσκηση 7.3: Forward Algorithm - Πίνακας	94
12.7.4Άσκηση 7.4: Viterbi Algorithm	94
12.7.5Άσκηση 7.5: Ερμηνεία HMM Παραμέτρων	95
12.8OLAP και Data Warehousing	96
12.8.1Άσκηση 8.1: OLAP Operations Ταξινόμηση	96
12.8.2Άσκηση 8.2: Roll-up και Drill-down Sequence	97
12.8.3Άσκηση 8.3: Slice vs Dice	97
12.8.4Άσκηση 8.4: Cube Calculations	98
12.8.5Άσκηση 8.5: Star Schema Design	99
12.8.6Άσκηση 8.6: OLAP Queries σε SQL	100
Σύνοψη Τύπων	103

Κεφάλαιο 1

ER Model & Database Design

Key Concepts - MEMORIZE!

ER Diagram Components:

- **Entity:** Rectangle - αντικείμενο/οντότητα (π.χ. Student, Course)
- **Attribute:** Oval - ιδιότητα οντότητας (π.χ. name, id)
- **Relationship:** Diamond - σύνδεση μεταξύ entities (π.χ. enrolls)
- **Primary Key:** Υπογραμμισμένο attribute

Cardinality Notation:

- 1:1 One-to-One (π.χ. Person - Passport)
- 1:N One-to-Many (π.χ. Department - Employee)
- M:N Many-to-Many (π.χ. Student - Course)

1.1 Βασικές Έννοιες

1.1.1 Entity Types

Strong vs Weak Entity

Έχει δικό του PK

Employee

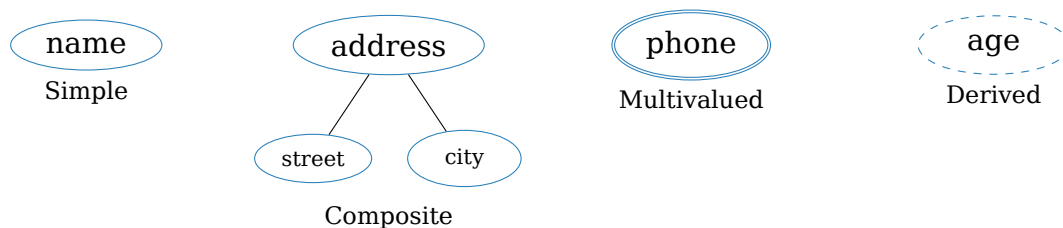
Strong Entity

Εξαρτάται από Strong

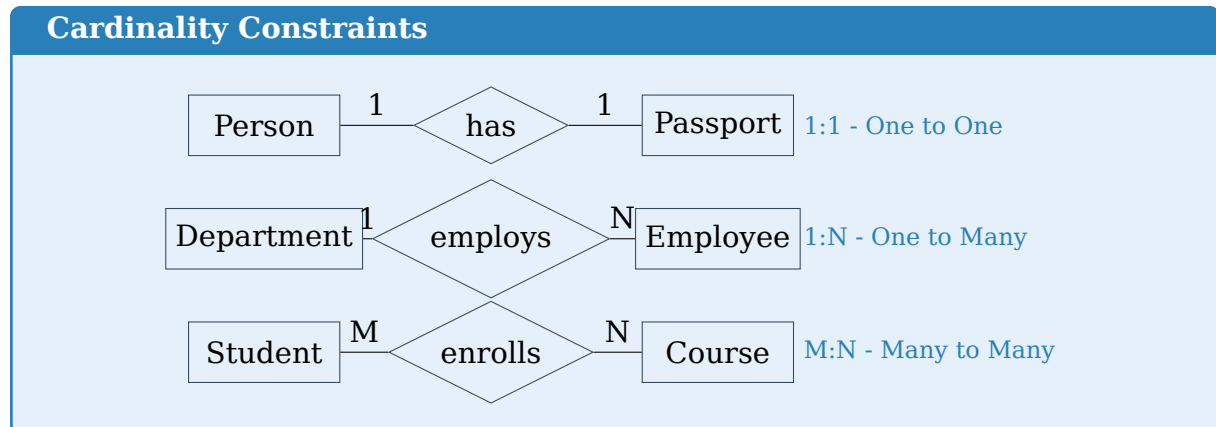
Dependent

Weak Entity

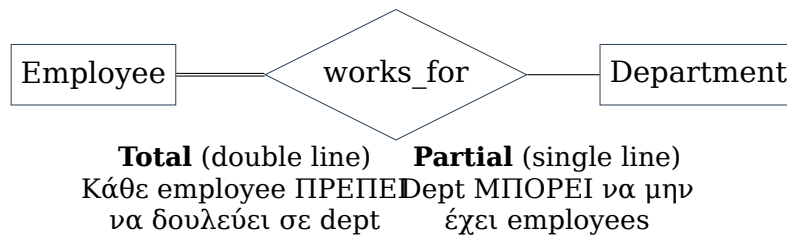
1.1.2 Attribute Types



1.2 Cardinality & Participation



1.2.1 Participation Constraints



1.3 Solved Exam Problems

1.3.1 2024-25 Exam Style: ER to Relational

Μετατροπή ER σε Relational Schema

Πρόβλημα: Δίνεται το παρακάτω ER diagram. Μετατρέψτε σε relational schema.

The ER diagram shows two entities: **Student** and **Course**. **Student** has a primary key sid and an attribute **name**. **Course** has a primary key cid and an attribute **title**. They are connected by an **enrolls** relationship with cardinalities **M** and **N**. The relationship has an attribute **grade**.

Λύση - Βήμα προς βήμα:
Step 1: Strong Entities → Tables

- Student(sid, name)
- Course(cid, title)

Step 2: M:N Relationship → New Table

- Enrolls(sid, cid, grade)
- Foreign Keys: sid → Student, cid → Course

Final Schema:

```

Student(sid, name)
Course(cid, title)
Enrolls(sid, cid, grade)
    FK: sid REFERENCES Student(sid)
    FK: cid REFERENCES Course(cid)

```

1.3.2 Cardinality Rules for Conversion

Cardinality	Conversion Rule	Example
1:1	FK σε οποιαδήποτε πλευρά	Person-Passport
1:N	FK στη N-πλευρά	Dept-Employee
M:N	Νέος πίνακας με 2 FKs	Student-Course

1.4 Common Mistakes**Συχνά Λάθη στις Εξετάσεις**

- M:N χωρίς νέο πίνακα:**
 - × Βάζω FK στη μία πλευρά
 - Δημιουργώ junction table με composite key
- Weak Entity conversion:**
 - × Μόνο το partial key ως PK
 - Composite key: (owner_PK + partial_key)
- Multivalued attributes:**
 - × Πολλές στήλες (phone1, phone2, ...)
 - Εξεχωριστός πίνακας με FK
- Total Participation:**
 - × Αγνοώ τη διπλή γραμμή
 - NOT NULL constraint στο FK

1.5 Quick Reference

ER → Relational Conversion Rules

1. **Strong Entity** → Table με όλα τα simple attributes
2. **Weak Entity** → Table με (owner_PK + partial_key) ως composite PK
3. **1:1 Relationship** → FK σε οποιαδήποτε πλευρά (προτίμηση: total participation)
4. **1:N Relationship** → FK στην N-πλευρά (many side)
5. **M:N Relationship** → Νέος πίνακας με PKs και των δύο entities
6. **Multivalued Attr** → Ξεχωριστός πίνακας (entity_PK, value)
7. **Composite Attr** → Flatten: κάθε component γίνεται στήλη
8. **Derived Attr** → ΔΕΝ αποθηκεύεται (υπολογίζεται)

Κεφάλαιο 2

Relational Model & Algebra

Key Operators - MEMORIZE!

Symbol	Name	Description	SQL Equivalent
σ	Selection	Επιλογή γραμμών (filter)	WHERE
π	Projection	Επιλογή στηλών	SELECT columns
$\cup$	Union	Ένωση συνόλων	UNION
$\cap$	Intersection	Τομή συνόλων	INTERSECT
$-$	Difference	Διαφορά συνόλων	EXCEPT
$\times$	Cartesian Product	Καρτεσιανό γινόμενο	FROM A, B
$\bowtie$	Natural Join	Ένωση με κοινές στήλες	NATURAL JOIN
$\bowtie_{\theta}$	Theta Join	Ένωση με συνθήκη	JOIN ON
ρ	Rename	Μετονομασία	AS

2.1 Βασικοί Τελεστές

2.1.1 Selection (σ)

Selection

Επιλογή γραμμών που ικανοποιούν συνθήκη.

$$\sigma_{condition}(R)$$

Conditions: =, $\neq$, <, >, $\leq$, $\geq$, AND, OR, NOT

Selection Example

Table Employee:

name	dept	salary
Alice	CS	50000
Bob	Math	45000
Carol	CS	60000

$$\sigma_{dept='CS'}(Employee) =$$

name	dept	salary
Alice	CS	50000
Carol	CS	60000

$$\sigma_{salary > 48000 \wedge dept = 'CS'}(Employee) =$$

name	dept	salary
Alice	CS	50000
Carol	CS	60000

2.1.2 Projection (π)

Projection

Επιλογή στηλών (και αφαίρεση duplicates).

$$\pi_{attr_1, attr_2, \dots}(R)$$

Projection Example

$$\pi_{name, dept}(Employee) =$$

name	dept
Alice	CS
Bob	Math
Carol	CS

$$\pi_{dept}(Employee) = \{CS, Math\} \text{ (duplicates removed!)}$$

2.1.3 Set Operations ($\cup$, $\cap$, $-$)

Compatibility Requirement

Για $\cup$, $\cap$, $-$: Οι πίνακες ΠΡΕΠΕΙ να έχουν **ίδιο schema** (ίδιος αριθμός στηλών, συμβατοί τύποι).

Set Operations

$$R: \{1, 2, 3\} \quad S: \{2, 3, 4\}$$

$$R \cup S = \{1, 2, 3, 4\} \quad R \cap S = \{2, 3\} \quad R - S = \{1\}$$

2.2 Join Operations

2.2.1 Natural Join ($\bowtie$)

Natural Join

Ενώνει πίνακες στις **κοινές στήλες** (ίδιο όνομα & τιμή).

$$R \bowtie S$$

Natural Join Example

	<u>sid</u>	<u>name</u>		<u>sid</u>	<u>course</u>
Student:	1	Alice	Enrolls:	1	CS101
	2	Bob		1	CS102
				3	CS101

$Student \bowtie Enrolls =$

<u>sid</u>	<u>name</u>	<u>course</u>
1	Alice	CS101
1	Alice	CS102

Σημείωση: $sid=3$ δεν εμφανίζεται (δεν υπάρχει στο *Student*)

2.2.2 Theta Join ($\bowtie_{\theta}$)

Theta Join

Join με arbitrary condition.

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

2.2.3 Outer Joins

Left Outer $\sqsupset$
Κρατάει ΟΛΕΣ
τις γραμμές του R
(NULL για unmatched)

Right Outer $\sqsupset$
Κρατάει ΟΛΕΣ
τις γραμμές του S
(NULL για unmatched)

Full Outer $\sqsupset$
Κρατάει ΟΛΕΣ
τις γραμμές
και των δύο

2.3 Solved Exam Problems

2.3.1 2024-25 Exam Question 6 Style

Query Translation: SQL → Relational Algebra

Tables:

- Student(sid, name, dept)
- Course(cid, title, credits)
- Enrolls(sid, cid, grade)

Query: Find names of CS students enrolled in courses with ≥ 4 credits.

SQL:

```

1 SELECT DISTINCT S.name
2 FROM Student S, Enrolls E, Course C
3 WHERE S.sid = E.sid
4       AND E.cid = C.cid
5       AND S.dept = 'CS'
6       AND C.credits >= 4;

```

Relational Algebra - Step by Step:

Step 1: Filter students from CS

$$R_1 = \sigma_{dept='CS'}(Student)$$

Step 2: Filter courses with ≥ 4 credits

$$R_2 = \sigma_{credits \geq 4}(Course)$$

Step 3: Join with Enrolls

$$R_3 = R_1 \bowtie Enrolls \bowtie R_2$$

Step 4: Project name

$$Result = \pi_{name}(R_3)$$

Final Answer (one expression):

$$\pi_{name}(\sigma_{dept='CS'}(Student) \bowtie Enrolls \bowtie \sigma_{credits \geq 4}(Course))$$

2.3.2 Division Operation

Division: "For All" Queries

Query: Find students enrolled in ALL courses.

Tables:

- Enrolls(sid, cid)
- Course(cid, title)

Solution:

$$\pi_{sid,cid}(Enrolls) \div \pi_{cid}(Course)$$

Division Rule:

$$R(A, B) \div S(B) = \text{tuples } a \text{ such that for ALL } b \in S, (a, b) \in R$$

Implementation using basic operators:

$$R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$$

Intuition: Start with all A values, then subtract those that are missing at least one B value.

2.4 Operator Precedence

Operator Precedence (Highest to Lowest)

1. σ, π, ρ (Unary operators)
2. $\times, \bowtie$ (Products and Joins)
3. $\cap$ (Intersection)
4. $\cup, -$ (Union and Difference)

2.5 Common Mistakes

Συχνά Λάθη στις Εξετάσεις

1. Projection removes duplicates:

- $\pi_{dept}(Employee)$ returns DISTINCT values!

2. Natural Join matches on ALL common columns:

- $\times R \bowtie S$ only on one column
- $\sqcap$ Matches on ALL columns with same name

3. Selection vs Projection confusion:

- σ = rows (horizontal slice)
- π = columns (vertical slice)

4. Set operations need compatible schemas:

- Cannot do $\pi_{name}(R) \cup \pi_{dept}(S)$ if different types

2.6 Quick Reference

Relational Algebra Cheat Sheet

Selection: $\sigma_{condition}(R)$ — Filter rows

Projection: $\pi_{cols}(R)$ — Choose columns (removes duplicates)

Natural Join: $R \bowtie S$ — Join on common column names

Theta Join: $R \bowtie_{\theta} S$ — Join with condition

Division: $R \div S$ — "For all" queries

Rename: $\rho_{newname}(R)$ — Change table/column name

SQL → RA Mapping:
 WHERE = σ , SELECT = π
 JOIN ON = $\bowtie_{\theta}$

Common Pattern:
 $\pi_{cols}(\sigma_{cond}(R \bowtie S))$

Κεφάλαιο 3

SQL Basics

Key SQL Syntax - MEMORIZE!

```
1 SELECT [DISTINCT] column1, column2, aggregate(column)
2 FROM table1 [JOIN table2 ON condition]
3 WHERE condition           -- Filter BEFORE grouping
4 GROUP BY column           -- Group rows
5 HAVING aggregate_condition -- Filter AFTER grouping
6 ORDER BY column [ASC|DESC]
7 LIMIT n;
```

Execution Order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

3.1 SELECT Basics

3.1.1 Column Selection

Basic SELECT

```
1 -- All columns
2 SELECT * FROM Employee;
3
4 -- Specific columns
5 SELECT name, salary FROM Employee;
6
7 -- With alias
8 SELECT name AS employee_name, salary * 12 AS annual
9 FROM Employee;
10
11 -- Remove duplicates
12 SELECT DISTINCT dept FROM Employee;
```

3.1.2 WHERE Clause

Operator	Example
=, <>, <, >	salary > 50000
BETWEEN	age BETWEEN 20 AND 30
IN	dept IN ('CS', 'Math')
LIKE	name LIKE 'A%'
IS NULL	manager IS NULL
AND, OR, NOT	dept='CS' AND salary > 50000

WHERE Examples

```

1 -- Pattern matching
2 SELECT * FROM Employee
3 WHERE name LIKE 'A%'; -- Starts with A
4
5 -- NULL check (NOT with =!)
6 SELECT * FROM Employee
7 WHERE manager IS NULL;
8
9 -- Range
10 SELECT * FROM Employee
11 WHERE salary BETWEEN 40000 AND 60000;

```

3.2 Aggregate Functions

Aggregate Functions

Function	Description	NULL Handling
COUNT(*)	Αριθμός γραμμών	Counts ALL
COUNT(col)	Αριθμός non-NULL τιμών	Ignores NULL
SUM(col)	Άθροισμα	Ignores NULL
AVG(col)	Μέσος όρος	Ignores NULL
MAX(col)	Μέγιστο	Ignores NULL
MIN(col)	Ελάχιστο	Ignores NULL

COUNT(*) vs COUNT(column)

```

1 -- Table: Employee(name, bonus)
2 -- Data: ('Alice', 100), ('Bob', NULL), ('Carol', 200)
3
4 SELECT COUNT(*) FROM Employee; -- Returns 3
5 SELECT COUNT(bonus) FROM Employee; -- Returns 2 (NULL ignored!)

```

3.3 GROUP BY & HAVING

GROUP BY vs HAVING

- **WHERE:** Φιλτράρει γραμμές ΠΙΝ το grouping
- **HAVING:** Φιλτράρει groups META το grouping

GROUP BY Example

Query: Average salary per department, only for departments with > 2 employees.

```

1 SELECT dept, AVG(salary) AS avg_salary, COUNT(*) AS emp_count
2 FROM Employee
3 WHERE salary > 30000          -- Filter rows first
4 GROUP BY dept                -- Then group
5 HAVING COUNT(*) > 2          -- Filter groups
6 ORDER BY avg_salary DESC;
```

SELECT with GROUP BY Rule

Στο SELECT μπορούν ΜΟΝΟ:

- Στήλες που είναι στο GROUP BY
- Aggregate functions

```

1 -- WRONG!
2 SELECT name, dept, AVG(salary) FROM Employee GROUP BY dept;
3 -- 'name' is not in GROUP BY!
4
5 -- CORRECT
6 SELECT dept, AVG(salary) FROM Employee GROUP BY dept;
```

3.4 Solved Exam Problems

3.4.1 2024-25 Exam Question 13 Style

Multi-table Query

Tables:

- Student(sid, name, dept)
- Course(cid, title, credits)
- Enrolls(sid, cid, grade)

Query: Find students with average grade > 8.0 in CS department.

```

1 SELECT S.name, AVG(E.grade) AS avg_grade
2 FROM Student S
3 JOIN Enrolls E ON S.sid = E.sid
4 WHERE S.dept = 'CS'
```

```

5 GROUP BY S.sid, S.name
6 HAVING AVG(E.grade) > 8.0
7 ORDER BY avg_grade DESC;

```

Step-by-step:

1. JOIN: Συνδέουμε Student με Enrolls
2. WHERE: Φιλτράρουμε CS students
3. GROUP BY: Ομαδοποιούμε ανά student
4. HAVING: Κρατάμε groups με AVG > 8.0
5. SELECT: Εμφανίζουμε name και average

3.4.2 2024-25 Exam Question 14 Style

Subquery Example

Query: Students who have NOT enrolled in any course.

```

1 -- Method 1: NOT IN
2 SELECT name FROM Student
3 WHERE sid NOT IN (SELECT DISTINCT sid FROM Enrolls);
4
5 -- Method 2: NOT EXISTS
6 SELECT name FROM Student S
7 WHERE NOT EXISTS (
8     SELECT 1 FROM Enrolls E WHERE E.sid = S.sid
9 );
10
11 -- Method 3: LEFT JOIN + NULL check
12 SELECT S.name
13 FROM Student S
14 LEFT JOIN Enrolls E ON S.sid = E.sid
15 WHERE E.sid IS NULL;

```

3.4.3 Finding Maximum with Details

Max Salary with Employee Details

Query: Find employee(s) with highest salary.

```

1 -- WRONG: Returns one row even if ties exist
2 SELECT * FROM Employee ORDER BY salary DESC LIMIT 1;
3
4 -- CORRECT: Handles ties
5 SELECT * FROM Employee
6 WHERE salary = (SELECT MAX(salary) FROM Employee);

```

3.5 Common Patterns

Pattern	SQL Template
Count per group	SELECT g, COUNT(*) FROM T GROUP BY g
Top N per group	WITH ranked AS (...) ROW_NUMBER() OVER
Self-join	FROM T t1, T t2 WHERE t1.x = t2.y
NOT IN pattern	WHERE x NOT IN (SELECT x FROM ...)

3.6 Common Mistakes

Συχνά Λάθη στις Εξετάσεις

1. NULL με = instead of IS:

- × WHERE bonus = NULL
- WHERE bonus IS NULL

2. HAVING χωρίς GROUP BY:

- HAVING πρέπει να χρησιμοποιείται με aggregate

3. Non-grouped columns στο SELECT:

- × SELECT name, SUM(x) GROUP BY dept
- SELECT dept, SUM(x) GROUP BY dept

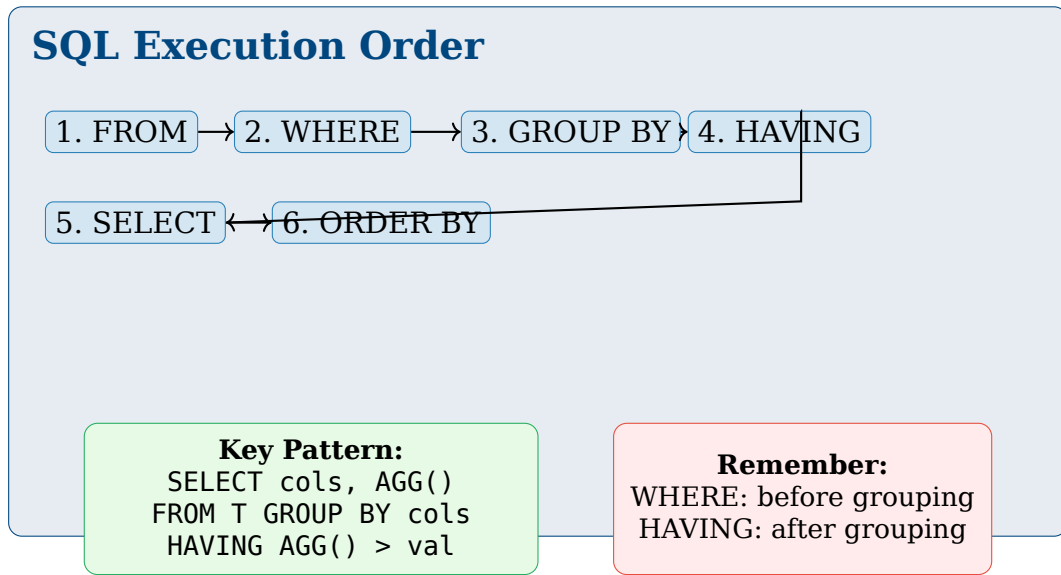
4. String comparison:

- × WHERE name = "Alice" (double quotes)
- WHERE name = 'Alice' (single quotes)

5. COUNT με NULL:

- COUNT(*) counts all rows
- COUNT(col) ignores NULL values

3.7 Quick Reference



Κεφάλαιο 4

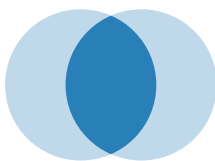
Advanced SQL

Key Concepts - MEMORIZE!

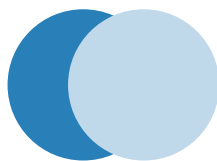
JOIN Types:

Type	Description
INNER JOIN	Μόνο matching rows
LEFT OUTER JOIN	Όλα από αριστερά + matching δεξιά (NULL αν δεν υπάρχει)
RIGHT OUTER JOIN	Όλα από δεξιά + matching αριστερά
FULL OUTER JOIN	Όλα από και τις δύο πλευρές
NATURAL JOIN	Auto-join on same column names
CROSS JOIN	Cartesian product (κάθε row με κάθε row)

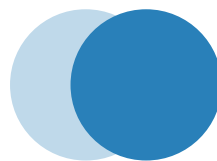
4.1 JOIN Types Visualization



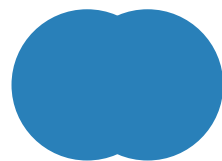
INNER JOIN
Only matching



LEFT JOIN
All left + matching right



RIGHT JOIN
All right + matching left



FULL OUTER
All from both

4.2 JOIN Examples

JOIN Comparison

Tables:

	eid	name		eid	dept
Employee:	1	Alice	Department:	1	CS
	2	Bob		2	Math
	3	Carol		4	Physics

INNER JOIN:

```
1 SELECT * FROM Employee E INNER JOIN Department D ON E.eid = D.eid;
```

Result: {(1, Alice, CS), (2, Bob, Math)}

LEFT JOIN:

```
1 SELECT * FROM Employee E LEFT JOIN Department D ON E.eid = D.eid;
```

Result: {(1, Alice, CS), (2, Bob, Math), (3, Carol, NULL)}

FULL OUTER JOIN:

```
1 SELECT * FROM Employee E FULL OUTER JOIN Department D ON E.eid = D.eid;
```

Result: {(1, Alice, CS), (2, Bob, Math), (3, Carol, NULL), (NULL, NULL, Physics)}

4.3 Self-Join

Self-Join: Finding Pairs

Query: Find employees who work in the same department.

```
1 SELECT e1.name, e2.name, e1.dept
2 FROM Employee e1, Employee e2
3 WHERE e1.dept = e2.dept
4 AND e1.eid < e2.eid; -- Avoid duplicates & self-pairs
```

4.4 Subqueries

4.4.1 Types of Subqueries

Scalar Subquery
Returns single value
WHERE x = (SELECT ...)

Table Subquery
Returns set of values
WHERE x IN (SELECT ...)

Correlated
References outer query
WHERE EXISTS (...)

4.4.2 EXISTS vs IN

EXISTS vs IN

```

1  -- Students enrolled in at least one course
2  -- Using IN
3  SELECT name FROM Student
4  WHERE sid IN (SELECT sid FROM Enrolls);
5
6  -- Using EXISTS (often faster for large tables)
7  SELECT name FROM Student S
8  WHERE EXISTS (SELECT 1 FROM Enrolls E WHERE E.sid = S.sid);

```

Key Difference:

- IN: Builds set, then checks membership
- EXISTS: Stops at first match (short-circuit)

4.5 Window Functions

Window Functions

```

1  function_name() OVER (
2      [PARTITION BY col]
3      [ORDER BY col]
4      [ROWS BETWEEN ... AND ...]
5  )

```

Common Functions:

- ROW_NUMBER(): Unique sequential number
- RANK(): Rank with gaps for ties
- DENSE_RANK(): Rank without gaps
- SUM(), AVG(), COUNT(): Running aggregates

ROW_NUMBER Example

Query: Top 3 highest paid employees per department.

```

1  WITH ranked AS (
2      SELECT name, dept, salary,
3              ROW_NUMBER() OVER (PARTITION BY dept
4                                  ORDER BY salary DESC) AS rn
5      FROM Employee
6  )
7  SELECT name, dept, salary
8  FROM ranked
9  WHERE rn <= 3;

```

RANK vs DENSE_RANK vs ROW_NUMBER

Data: Scores = [100, 100, 90, 80]

Score	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
90	3	3	2
80	4	4	3

Key Differences:

- ROW_NUMBER: Always unique, arbitrary for ties
- RANK: Same rank for ties, **skips** next ranks (gap after tie)
- DENSE_RANK: Same rank for ties, **no gaps**

```

1 SELECT score,
2     ROW_NUMBER() OVER (ORDER BY score DESC) AS row_num,
3     RANK() OVER (ORDER BY score DESC) AS rank,
4     DENSE_RANK() OVER (ORDER BY score DESC) AS dense_rank
5 FROM Scores;
```

4.6 Common Table Expressions (CTE)

CTE with WITH

```

1 WITH high_earners AS (
2     SELECT * FROM Employee WHERE salary > 50000
3 ),
4 cs_employees AS (
5     SELECT * FROM high_earners WHERE dept = 'CS'
6 )
7 SELECT * FROM cs_employees;
```

4.7 Solved Exam Problems

4.7.1 2024-25 Exam Question 7 Style: Join Algorithm

Hash Join vs Merge Join vs Nested Loop

Question: Which join algorithm is best for each scenario?

Algorithm	Best When	Complexity
Nested Loop	Small tables, or indexed inner	$O(n \cdot m)$
Hash Join	Equality joins, one table fits in memory	$O(n + m)$
Merge Join	Both tables sorted on join key	$O(n + m)$

Decision Tree:

1. Sorted on join key? → **Merge Join**
2. Equality condition + one fits memory? → **Hash Join**
3. Small tables or index available? → **Nested Loop**

4.7.2 Complex Query with Multiple Joins**Multi-table Join****Tables:**

- Student(sid, name, advisor_id)
- Professor(pid, name, dept)
- Course(cid, title, prof_id)
- Enrolls(sid, cid, grade)

Query: Find students whose advisor teaches a course they're enrolled in.

```

1 SELECT DISTINCT S.name
2 FROM Student S
3 JOIN Professor P ON S.advisor_id = P.pid
4 JOIN Course C ON P.pid = C.prof_id
5 JOIN Enrolls E ON S.sid = E.sid AND C.cid = E.cid;
```

4.8 Common Mistakes**Συχνά Λάθη στις Εξετάσεις****1. LEFT JOIN με WHERE condition on right table:**

```

1 -- WRONG: Converts to INNER JOIN!
2 SELECT * FROM A LEFT JOIN B ON A.id = B.id
3 WHERE B.value = 'x';
4
5 -- CORRECT: Put condition in ON clause
6 SELECT * FROM A LEFT JOIN B ON A.id = B.id AND B.value = 'x';
```

2. NATURAL JOIN surprises:

- Matches on ALL columns with same name

- Can fail silently if column names change

3. **Correlated subquery performance:**

- Executes once per outer row
- Consider JOIN for better performance

4.9 Quick Reference

Advanced SQL Patterns

Find unmatched rows:

`SELECT * FROM A LEFT JOIN B ON ... WHERE B.id IS NULL`

Top N per group:

`WITH r AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY ...) AS rn)
SELECT * FROM r WHERE rn <= N`

Running total:

`SUM(x) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)`

Exists check:

`WHERE EXISTS (SELECT 1 FROM ... WHERE ...)`

INNER: Only matches
LEFT: All left + matches

Hash Join: Equality, memory
Merge Join: Sorted data

Κεφάλαιο 5

Query Processing & Optimization

Key Concepts - MEMORIZE!

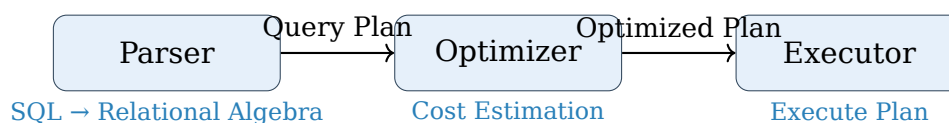
Evaluation Methods:

Method	Description
Materialized	Κάθε operator υπολογίζει πλήρως και αποθηκεύει αποτέλεσμα πριν το δώσει στον επόμενο
Pipelined	Tuples περνούν στον επόμενο operator αμέσως μόλις παραχθούν (streaming)

Join Algorithms Complexity:

Nested Loop Join	$O(n \times m)$
Hash Join	$O(n + m)$
Merge Join	$O(n + m)$ (sorted input)

5.1 Query Evaluation Pipeline



5.2 Materialized vs Pipelined Evaluation

5.2.1 Materialized Evaluation

Materialized Evaluation

Κάθε operation υπολογίζει **πλήρως** το αποτέλεσμά του και το αποθηκεύει (temporary table) πριν περάσει στην επόμενη operation.

Πλεονεκτήματα:

- Απλή υλοποίηση
- Μπορεί να επαναχρησιμοποιηθεί αποτέλεσμα

Μειονεκτήματα:

- Χρειάζεται μεγάλο disk I/O
- Καθυστέρηση μέχρι να τελειώσει κάθε βήμα

5.2.2 Pipelined Evaluation**Pipelined Evaluation**

Τα tuples περνούν στον επόμενο operator **αμέσως** μόλις παραχθούν (on-the-fly).

Πλεονεκτήματα:

- Μειωμένο disk I/O
- Χαμηλή latency (αρχίζει να επιστρέφει αποτελέσματα νωρίς)
- Μικρότερη χρήση μνήμης

Μειονεκτήματα:

- Πιο περίπλοκη υλοποίηση
- Δεν γίνεται πάντα (π.χ. sorting, hash build)

Materialized

Store complete result
before next step

Pipelined

Pass tuples directly
(streaming)

5.3 Join Algorithms**5.3.1 Nested Loop Join****Nested Loop Join**

```

1 for each tuple r in R:
2   for each tuple s in S:
3     if r.key == s.key:
4       emit (r, s)
  
```

Cost: $O(|R| \times |S|)$ comparisons

Best when:

- One table is very small
- Index exists on inner table

5.3.2 Hash Join

Hash Join

Phase 1: Build

```

1 for each tuple r in R (smaller table):
2     insert r into hash_table on r.key

```

Phase 2: Probe

```

1 for each tuple s in S:
2     for each r in hash_table.lookup(s.key):
3         emit (r, s)

```

Cost: $O(|R| + |S|)$

Best when:

- Equality join condition
- Smaller table fits in memory

5.3.3 Merge Join (Sort-Merge Join)

Merge Join

Requirement: Both tables sorted on join key

```

1 // Both R, S sorted on join key
2 r = first(R), s = first(S)
3 while not end of R and not end of S:
4     if r.key < s.key: r = next(R)
5     else if r.key > s.key: s = next(S)
6     else: // r.key == s.key
7         emit all matching pairs
8         advance pointers

```

Cost: $O(|R| + |S|)$ (if already sorted) + sorting cost

Best when:

- Data already sorted (index)
- Result of ORDER BY

5.3.4 Join Algorithm Comparison

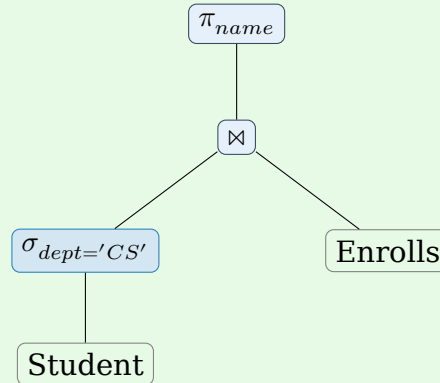
Algorithm	Best Case	Requirements
Nested Loop	Small inner table	Index on inner
Hash Join	Equality join	Memory for hash table
Merge Join	Sorted data	Pre-sorted input

5.4 Solved Exam Problems

5.4.1 2024-25 Exam Question 5 Style

Pipelined vs Materialized

Question: Given the query plan below, identify which operations can be pipelined.



Answer:

- σ (Selection): **Pipelined** - φιλτράρει tuples on-the-fly
- $\bowtie$ (Join): Εξαρτάται από algorithm
 - Hash Join build phase: **Blocking**
 - Nested Loop: **Pipelined**
- π (Projection): **Pipelined** - transforms tuples immediately

5.4.2 Cost Estimation

Disk I/O Cost

Given:

- Table R: 1000 pages, 10000 tuples
- Table S: 500 pages, 5000 tuples
- Memory: 50 buffer pages

Question: Calculate I/O cost for Nested Loop Join.

Solution:

$$\text{Cost} = |R| + |R| \times |S| = 1000 + 1000 \times 500 = 501000 \text{ I/Os}$$

With Block Nested Loop (using memory):

$$\text{Cost} = |R| + \lceil |R|/B \rceil \times |S| = 1000 + \lceil 1000/48 \rceil \times 500 = 1000 + 21 \times 500 = 11500 \text{ I/Os}$$

(B = buffer pages for R = 50 - 2 = 48)

5.5 Query Optimization Rules

Optimization Heuristics

1. **Push selections down:** Φιλτράρισε νωρίς, μείωσε δεδομένα
2. **Push projections down:** Κράτα μόνο needed columns
3. **Combine selections:** $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c1 \wedge c2}(R)$
4. **Reorder joins:** Βάλε μικρότερο πίνακα ως outer
5. **Use indexes:** Avoid full table scans

5.6 Common Mistakes

Συχνά Λάθη στις Εξετάσεις

1. **Hash Join is always best:**
 - × Hash Join πάντα καλύτερο
 - Χρειάζεται equality condition + memory
2. **Pipelined = παντού:**
 - Sort, Hash build, Aggregation: **Blocking** operations
 - Selection, Projection: Pipelined
3. **Merge Join χωρίς sorting:**
 - Πρέπει data να είναι sorted on join key
 - Sorting cost μπορεί να είναι significant

5.7 Quick Reference

Query Processing Summary

Blocking Operations:

- Sort, Hash table build, Aggregation (needs all input)

Pipelined Operations:

- Selection (σ), Projection (π), Nested Loop (outer)

Join Selection:

- Equality + Memory $\rightarrow$ Hash Join
- Sorted data $\rightarrow$ Merge Join
- Small table / Index $\rightarrow$ Nested Loop

Materialized: Store full result
Pipelined: Stream tuples

Optimize by:
 Push σ down, use indexes

Κεφάλαιο 6

Data Preprocessing

Key Concepts - MEMORIZE!

Attribute Types:

Type	Operations	Example	Properties
Nominal	$=, \neq$	Color, Gender	Categories, no order
Ordinal	$=, \neq, <, >$	Rating (1-5)	Ordered categories
Interval	$+, -$	Temperature (°C)	Meaningful differences
Ratio	$\times, \div$	Height, Weight	Absolute zero exists

Distance Formulas:

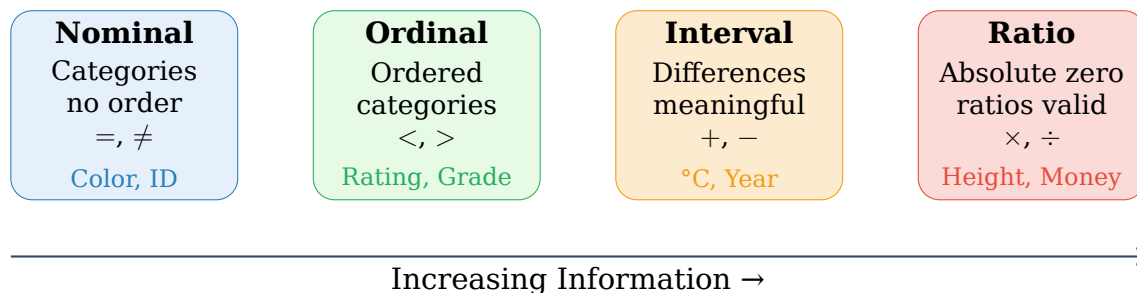
$$\text{Euclidean: } d = \sqrt{\sum_i (x_i - y_i)^2}$$

$$\text{Manhattan: } d = \sum_i |x_i - y_i|$$

$$\text{Cosine Similarity: } \cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

6.1 Attribute Types (ΕΞΕΤΑΣΤΙΚΟ ΘΕΜΑ!)

6.1.1 NOIR Scale



Attribute Type Classification

Question (2024-25 Exam Style): Classify each attribute:

Attribute	Type	Reasoning
Student ID	Nominal	Just identifier, no meaningful order
Letter Grade (A-F)	Ordinal	Ordered but no meaningful differences
Temperature (°C)	Interval	0°C is not "no temperature"
Height (cm)	Ratio	0 cm = no height, ratios meaningful
ZIP Code	Nominal	Numbers used as categories
Year	Interval	Year 0 is arbitrary
Age	Ratio	0 = no age, can say "twice as old"

Common Confusion

Interval vs Ratio:

- Temperature in **Celsius/Fahrenheit**: Interval (0° is not "no heat")
- Temperature in **Kelvin**: Ratio (0K = absolute zero)
- **Test**: Can you say "twice as much"? If yes → Ratio

6.2 Distance Measures

6.2.1 Euclidean Distance

Euclidean Distance (L2)

$$d_{Euclidean}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Properties:

- Sensitive to scale (χρειάζεται normalization)
- Works for continuous attributes

6.2.2 Manhattan Distance

Manhattan Distance (L1)

$$d_{Manhattan}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

Properties:

- Less sensitive to outliers than Euclidean
- "City block" distance

6.2.3 Cosine Similarity

Cosine Similarity

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \cdot \sqrt{\sum_i y_i^2}}$$

Properties:

- Range: $[-1, 1]$ (or $[0, 1]$ for non-negative vectors)
- Measures **direction**, ignores magnitude
- Common for text/document similarity

6.2.4 Jaccard Similarity

Jaccard Similarity

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Properties:

- Range: $[0, 1]$ (1 = identical sets, 0 = no overlap)
- Best for **binary/set data**
- Common for document comparison, recommendation systems

Jaccard Distance: $d_J(A, B) = 1 - J(A, B)$

Jaccard Similarity Calculation

Sets: $A = \{1, 2, 3, 4\}$, $B = \{2, 3, 5, 6\}$

$$A \cap B = \{2, 3\} \Rightarrow |A \cap B| = 2$$

$$A \cup B = \{1, 2, 3, 4, 5, 6\} \Rightarrow |A \cup B| = 6$$

$$J(A, B) = \frac{2}{6} = \frac{1}{3} \approx 0.33$$

Distance Calculation

Points: $A = (1, 2)$, $B = (4, 6)$

Euclidean:

$$d = \sqrt{(4-1)^2 + (6-2)^2} = \sqrt{9+16} = \sqrt{25} = 5$$

Manhattan:

$$d = |4-1| + |6-2| = 3+4 = 7$$

Cosine Similarity:

$$\cos(\theta) = \frac{1 \times 4 + 2 \times 6}{\sqrt{1^2 + 2^2} \cdot \sqrt{4^2 + 6^2}} = \frac{4+12}{\sqrt{5} \cdot \sqrt{52}} = \frac{16}{\sqrt{260}} \approx 0.99$$

6.3 Data Discretization

6.3.1 Equal-Width Partitioning

Equal-Width Binning

Διαίρεση του range σε k bins με **ίσο πλάτος**.

$$\text{Width} = \frac{\max - \min}{k}$$

Bin i : $[\min + (i - 1) \times \text{width}, \min + i \times \text{width})$

6.3.2 Equal-Frequency (Equal-Depth) Partitioning

Equal-Frequency Binning

Διαίρεση ώστε κάθε bin να έχει **ίσο αριθμό records**.

$$\text{Records per bin} = \frac{n}{k}$$

Discretization - 2024-25 Exam Q16 Style

Data: 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34

Equal-Width (3 bins):

- Range: $34 - 4 = 30$, Width = $30/3 = 10$
- Bin 1 [4, 14): 4, 8, 9
- Bin 2 [14, 24): 15, 21, 21
- Bin 3 [24, 34]: 24, 25, 26, 28, 29, 34

Equal-Frequency (3 bins):

- 12 values / 3 bins = 4 per bin
- Bin 1: 4, 8, 9, 15
- Bin 2: 21, 21, 24, 25
- Bin 3: 26, 28, 29, 34

6.4 Data Normalization

Min-Max Normalization

$$x' = \frac{x - \min}{\max - \min}$$

Range: [0, 1]

Z-Score Standardization

$$z = \frac{x - \mu}{\sigma}$$

Mean=0, Std=1

6.5 Missing Value Handling

Method	When to Use	Risk
Delete rows	Few missing values	Loss of data
Mean/Median imputation	Numerical, MCAR	Reduces variance
Mode imputation	Categorical	May bias data
Regression imputation	Correlated features	Complex

6.6 Common Mistakes

Συχνά Λάθη στις Εξετάσεις

1. Interval vs Ratio confusion:

- Temperature °C: **Interval** (20°C is NOT "twice as hot" as 10°C)
- Height cm: **Ratio** (200cm IS twice 100cm)

2. Equal-Width bin boundaries:

- Include min, exclude max in all but last bin
- Last bin: $[\cdot, \max]$

3. Cosine vs Euclidean:

- Cosine: direction only (good for text)
- Euclidean: considers magnitude

6.7 Quick Reference

Data Preprocessing Cheat Sheet

Attribute Types (NOIR):

Nominal → Ordinal → Interval → Ratio (increasing info)

Distance Measures:

Euclidean: $\sqrt{\sum (x_i - y_i)^2}$ Manhattan: $\sum |x_i - y_i|$

Cosine: $\frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$ (direction only)

Discretization:

Equal-Width: width = $\frac{\max - \min}{k}$

Equal-Frequency: n/k records per bin

Ratio vs Interval Test:

Can say "twice as much"?
Yes → Ratio, No → Interval

Normalize When:

Features have different scales
Using distance-based methods

Κεφάλαιο 7

Data Streams & Bloom Filters

Key Concepts - MEMORIZE!

Data Stream Characteristics:

- Continuous, unbounded data flow
- Cannot store all data in memory
- Must process in single pass (or limited passes)
- Approximate answers often acceptable

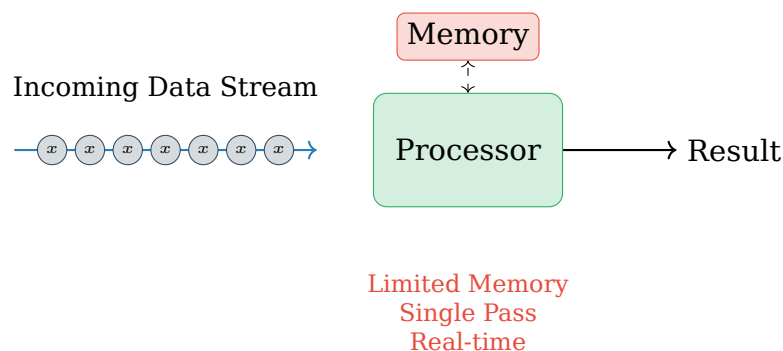
Bloom Filter:

False Positive Probability: $p \approx (1 - e^{-kn/m})^k$

Optimal # hash functions: $k = \frac{m}{n} \ln 2$

where m = bits, n = elements, k = hash functions

7.1 Stream Processing Model



7.2 Sampling Techniques

7.2.1 Reservoir Sampling

Reservoir Sampling

Goal: Select k random items from stream of unknown size.

```

1 // Keep k items uniformly at random
2 reservoir = first k items
3 for i = k+1 to n:
4     j = random(1, i)
5     if j <= k:
6         reservoir[j] = stream[i]
7 return reservoir

```

Property: Each item has equal probability k/n of being selected.

7.2.2 Sliding Window

Sliding Window

Διατήρηση των τελευταίων N στοιχείων (ή χρονικό παράθυρο).

Types:

- **Count-based:** Last N elements
- **Time-based:** Last T time units

7.3 Bloom Filters

7.3.1 Structure

Bloom Filter

Probabilistic data structure για fast membership testing.

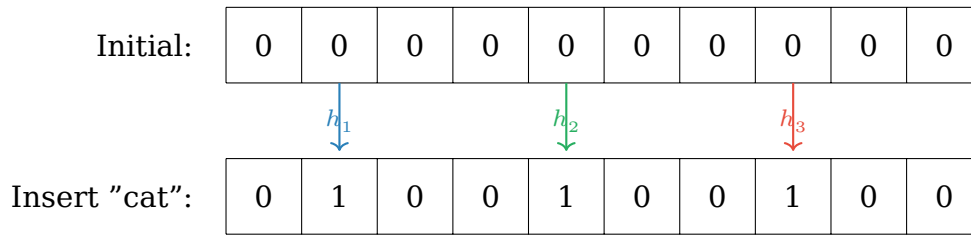
Components:

- Bit array of size m
- k independent hash functions

Properties:

- No false negatives (αν λέει "no", σίγουρα δεν υπάρχει)
- Possible false positives (αν λέει "yes", μπορεί να μην υπάρχει)
- Cannot delete elements (standard Bloom filter)

7.3.2 Operations



Bloom Filter Operations

Insert(x):

```

1 for i = 1 to k:
2   bit_array[hash_i(x) mod m] = 1

```

Query(x):

```

1 for i = 1 to k:
2   if bit_array[hash_i(x) mod m] == 0:
3     return "NOT in set" // Definitely not
4 return "MAYBE in set" // Might be false positive

```

7.3.3 False Positive Probability

False Positive Rate

After inserting n elements into m bits with k hash functions:

$$p_{FP} \approx (1 - e^{-kn/m})^k$$

Optimal number of hash functions:

$$k_{opt} = \frac{m}{n} \ln 2 \approx 0.693 \frac{m}{n}$$

Required bits per element for target FP rate:

$$\frac{m}{n} = -\frac{\ln p_{FP}}{(\ln 2)^2} \approx -1.44 \log_2 p_{FP}$$

Bloom Filter Calculation

Given: Want to store $n = 1000$ elements with FP rate $\leq 1\%$

Calculate required bits:

$$\frac{m}{n} = -1.44 \log_2(0.01) = -1.44 \times (-6.64) \approx 9.6$$

$$m = 9.6 \times 1000 \approx 10000 \text{ bits} \approx 1.25 \text{ KB}$$

Optimal hash functions:

$$k = 0.693 \times 9.6 \approx 7 \text{ hash functions}$$

7.4 Count-Min Sketch

Count-Min Sketch

Data structure για approximate frequency counting σε streams.

Structure:

- 2D array: d rows $\times$ w columns
- d hash functions (one per row)

Operations:

- **Update(x):** Increment counter at $h_i(x)$ for each row i
- **Query(x):** Return $\min_i \text{counter}[i][h_i(x)]$

Property: Never underestimates, may overestimate

7.5 Flajolet-Martin Algorithm

Flajolet-Martin Algorithm

Algorithm for estimating the number of **distinct elements** in a data stream.

Key Idea:

- Hash each element to a binary string
- Track R = maximum number of trailing zeros seen
- Estimate: 2^R distinct elements

Flajolet-Martin Estimation

$$\text{Estimated distinct elements} = 2^R$$

where R = maximum number of trailing zeros in hash values seen.

Intuition:

- Probability of r trailing zeros = $1/2^r$
- If we see r trailing zeros, likely $\approx 2^r$ distinct elements
- Use multiple hash functions and combine (average) for accuracy

Flajolet-Martin Example

Stream elements and their hash values:

Element	Hash (binary)	Trailing Zeros
a	101100	1
b	110010	1
c	10100	2
d	1000	3
e	11111	0

$$R = \max(1, 1, 2, 3, 0) = 3$$

Estimate: $2^3 = 8$ distinct elements

(Actual: 5 distinct elements — approximate!)

7.6 Solved Exam Problems

Bloom Filter Query

Given:

- Bloom filter with $m = 10$ bits, $k = 3$ hash functions
- Elements inserted: "apple", "banana"
- Hash values:
apple: $h_1 = 2, h_2 = 5, h_3 = 8$
banana: $h_1 = 1, h_2 = 5, h_3 = 9$

Bit array after insertions:

0	1	1	0	0	1	0	0	1	1
Index: 0	1	2	3	4	5	6	7	8	9

Query "cherry": $h_1 = 2, h_2 = 6, h_3 = 8$

- Check bit 2: 1 ☐
- Check bit 6: 0 ☐
- Result: **NOT in set** (definitely correct)

Query "date": $h_1 = 1, h_2 = 5, h_3 = 9$

- Check bit 1: 1 ☐
- Check bit 5: 1 ☐
- Check bit 9: 1 ☐
- Result: **MAYBE in set** (false positive! "date" was never inserted)

7.7 Common Mistakes

Συχνά Λάθη στις Εξετάσεις

1. False negatives in Bloom filters:

- × Bloom filter can have false negatives
- ☐ ONLY false positives, NEVER false negatives

2. Deletion from Bloom filter:

- × Just set bits to 0
- ☐ Cannot delete (use Counting Bloom Filter instead)

3. More hash functions = always better:

- × Use as many hash functions as possible
- Optimal k depends on m/n ; too many increases FP

7.8 Quick Reference

Data Streams Summary

Bloom Filter:

- No false negatives, possible false positives
- Cannot delete elements
- $p_{FP} \approx (1 - e^{-kn/m})^k$

Reservoir Sampling:

- Uniform sample from stream of unknown size
- Each item has k/n probability

Sliding Window:

- Count-based or Time-based

Bloom Filter Answer:

"No" = Definitely not in set
 "Yes" = Maybe in set

Optimal Hash Functions:

$$k = 0.693 \times (m/n)$$

Κεφάλαιο 8

Discrete Sequences & HMMs

Key Formulas - MEMORIZE!

HMM Components:

- $S = \{s_1, \dots, s_N\}$: Hidden states
- $V = \{v_1, \dots, v_M\}$: Observable symbols
- $A = [a_{ij}]$: Transition probabilities, $a_{ij} = P(s_j | s_i)$
- $B = [b_j(k)]$: Emission probabilities, $b_j(k) = P(v_k | s_j)$
- $\pi = [\pi_i]$: Initial state distribution

Viterbi Algorithm:

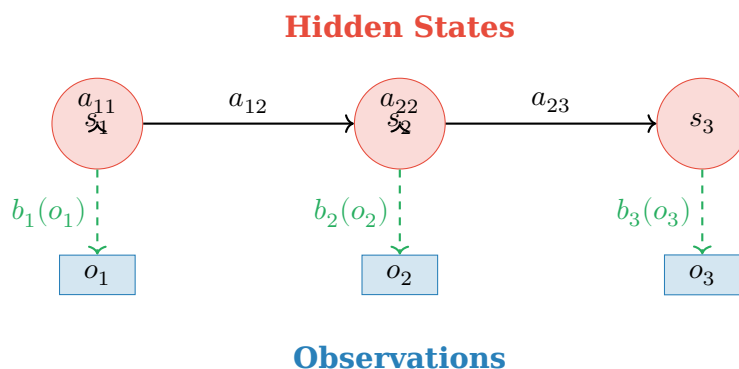
$$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

Sequence Support:

$$\text{Support}(S) = \frac{|\text{sequences containing } S|}{|\text{total sequences}|}$$

8.1 Hidden Markov Models (HMMs)

8.1.1 Model Structure



8.1.2 Three Fundamental Problems

1. Evaluation
 $P(O|\lambda)$
 Given model,
 what's obs probability?
 Forward Algorithm

2. Decoding
 Best state sequence
 given observations
 Viterbi Algorithm

3. Learning
 Estimate model
 parameters from data
 Baum-Welch (EM)

8.2 Viterbi Algorithm (EXAM QUESTION!)

Viterbi Algorithm

Goal: Find most likely state sequence given observations.

Initialization:

$$\delta_1(i) = \pi_i \cdot b_i(o_1), \quad \psi_1(i) = 0$$

Recursion:

$$\delta_t(j) = \max_{i=1}^N [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

$$\psi_t(j) = \arg \max_{i=1}^N [\delta_{t-1}(i) \cdot a_{ij}]$$

Termination:

$$P^* = \max_{i=1}^N \delta_T(i), \quad q_T^* = \arg \max_{i=1}^N \delta_T(i)$$

Backtracking:

$$q_t^* = \psi_{t+1}(q_{t+1}^*)$$

Viterbi - 2024-25 Exam Q12 Style

HMM Model:

- States: $S = \{H, L\}$ (High, Low)
- Observations: $V = \{A, B\}$
- Initial: $\pi_H = 0.6, \pi_L = 0.4$

Transition Matrix A :

	H	L
H	0.7	0.3
L	0.4	0.6

Emission Matrix B :

	A	B
H	0.2	0.8
L	0.5	0.5

Observation sequence: $O = (B, A)$

Step 1: Initialization ($t = 1$, observe B)

$$\delta_1(H) = \pi_H \cdot b_H(B) = 0.6 \times 0.8 = 0.48$$

$$\delta_1(L) = \pi_L \cdot b_L(B) = 0.4 \times 0.5 = 0.20$$

Step 2: Recursion ($t = 2$, observe A)

$$\begin{aligned}
\delta_2(H) &= \max\{\delta_1(H) \cdot a_{HH}, \delta_1(L) \cdot a_{LH}\} \cdot b_H(A) \\
&= \max\{0.48 \times 0.7, 0.20 \times 0.4\} \times 0.2 \\
&= \max\{0.336, 0.08\} \times 0.2 = 0.336 \times 0.2 = \mathbf{0.0672} \\
\psi_2(H) &= H
\end{aligned}$$

$$\begin{aligned}
\delta_2(L) &= \max\{\delta_1(H) \cdot a_{HL}, \delta_1(L) \cdot a_{LL}\} \cdot b_L(A) \\
&= \max\{0.48 \times 0.3, 0.20 \times 0.6\} \times 0.5 \\
&= \max\{0.144, 0.12\} \times 0.5 = 0.144 \times 0.5 = \mathbf{0.072} \\
\psi_2(L) &= H
\end{aligned}$$

Step 3: Termination

$$P^* = \max\{0.0672, 0.072\} = 0.072, \quad q_2^* = L$$

Step 4: Backtracking

$$q_1^* = \psi_2(L) = H$$

Answer: Most likely sequence is **H → L**

8.3 Sequential Pattern Mining

8.3.1 Definitions

Sequence Terminology

- **Sequence:** Ordered list of itemsets, e.g., $\langle\{A\}, \{B, C\}, \{D\}\rangle$
- **Subsequence:** α is subsequence of β if all itemsets of α appear in β in order
- **Support:** Fraction of sequences containing the pattern

Subsequence - 2024-25 Exam Q11 Style

Given sequences:

SID	Sequence
1	$\langle\{A\}, \{A, B\}, \{C\}\rangle$
2	$\langle\{A, B\}, \{C\}\rangle$
3	$\langle\{A\}, \{B\}, \{C\}\rangle$
4	$\langle\{A\}, \{C\}\rangle$

Question: Is $\langle\{A\}, \{C\}\rangle$ a subsequence of Sequence 1?

Answer: Yes! $\{A\} \subseteq \{A\}$ and $\{C\} \subseteq \{C\}$, appearing in order.

Support of $\langle\{A\}, \{C\}\rangle$:

- Seq 1: $\square$ ($\{A\}$ then $\{C\}$)
- Seq 2: $\square$ ($\{A, B\} \supseteq \{A\}$ then $\{C\}$)

- Seq 3: □
- Seq 4: □

Support = $4/4 = 100\%$

8.3.2 GSP Algorithm (Generalized Sequential Pattern)

GSP Overview

Similar to Apriori but for sequences:

1. Find frequent 1-sequences
2. Generate candidate k -sequences from $(k - 1)$ -sequences
3. Prune using Apriori property
4. Count support and keep frequent

8.4 Common Mistakes

Συχνά Λάθη στις Εξετάσεις

1. Viterbi: Forgetting to multiply by emission:

$$\times \delta_t(j) = \max[\delta_{t-1}(i) \cdot a_{ij}]$$

$$\square \delta_t(j) = \max[\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

2. Subsequence ordering:

- Itemsets must appear in the SAME ORDER
- But items within itemset can be in any order

3. HMM emission vs transition:

- a_{ij} : State $i \rightarrow$ State j (transition)
- $b_j(o)$: State j emits observation o (emission)

8.5 Quick Reference

HMM & Sequences Cheat Sheet

HMM Components:

- States (S), Observations (V), Transitions (A), Emissions (B), Initial (π)

Viterbi:

$$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

Backtrack using $\psi_t(j)$

Sequential Patterns:

Subsequence: ordered itemsets appearing in sequence

Support: fraction of sequences containing pattern

Viterbi Steps:

1. Init: $\pi \cdot b(o_1)$
2. Recurse: $\max \text{ prev} \times \text{trans} \times \text{emit}$
3. Backtrack

Three HMM Problems:

Evaluation: Forward
Decoding: Viterbi
Learning: Baum-Welch

Κεφάλαιο 9

OLAP & Data Warehousing

Key Operations - MEMORIZE!

OLAP Operations:

Operation	Description
Roll-up	Aggregation: λεπτομέρεια → σύνοψη (π.χ. day → month)
Drill-down	Disaggregation: σύνοψη → λεπτομέρεια (π.χ. year → quarter)
Slice	Selection on ONE dimension (π.χ. time = 'Q1')
Dice	Selection on MULTIPLE dimensions
Pivot	Rotate cube (swap dimensions)

Schema Types:

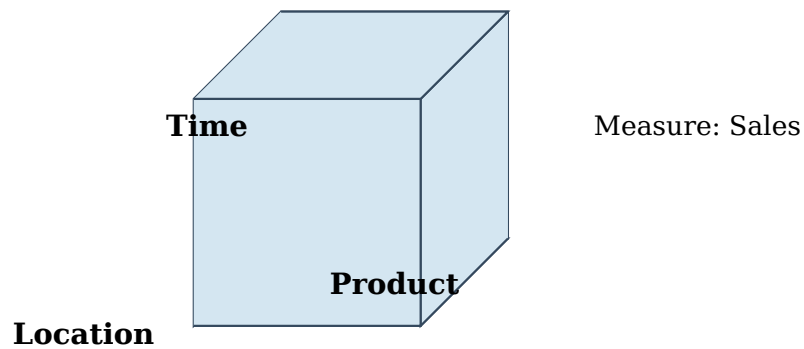
- **Star Schema:** Central fact table + dimension tables (denormalized)
- **Snowflake Schema:** Dimension tables are normalized

9.1 Data Warehouse Concepts

9.1.1 OLTP vs OLAP

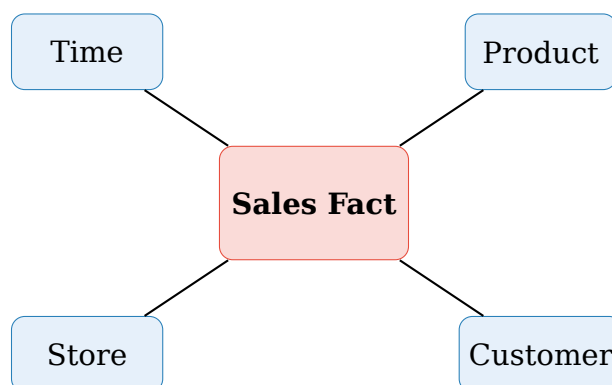
Aspect	OLTP	OLAP
Purpose	Daily operations	Analysis/Reporting
Data	Current, detailed	Historical, summarized
Queries	Simple, frequent	Complex, ad-hoc
Updates	Many, small	Periodic batch loads
Users	Clerks, customers	Analysts, managers

9.1.2 Data Cube Structure



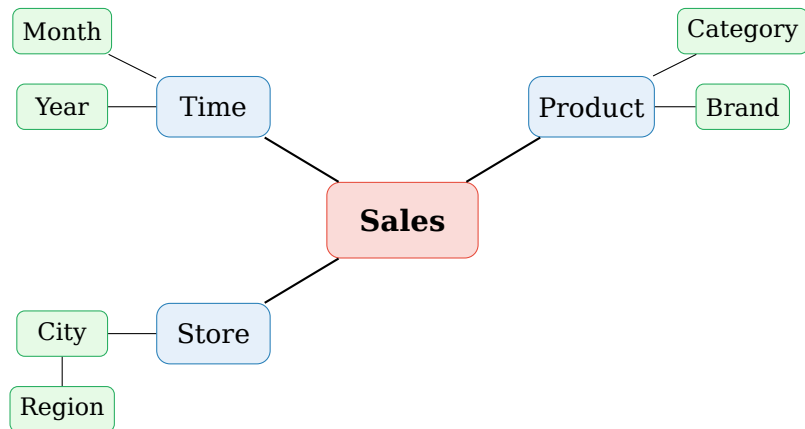
9.2 Star vs Snowflake Schema

9.2.1 Star Schema



- **Advantages:** Simple queries, fewer joins
- **Disadvantage:** Data redundancy in dimensions

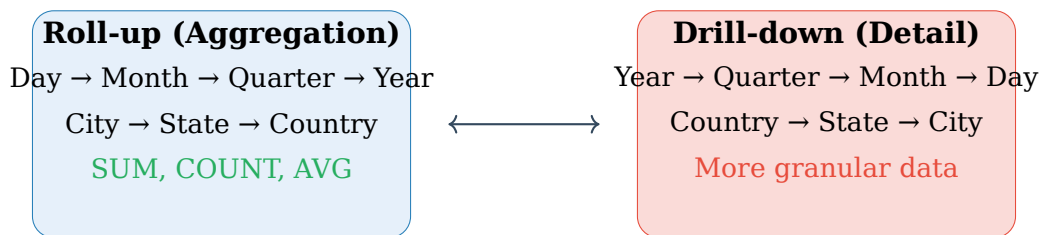
9.2.2 Snowflake Schema



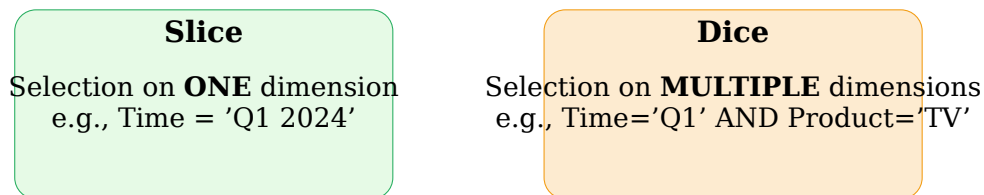
- **Advantage:** Less redundancy, normalized
- **Disadvantage:** More complex queries, more joins

9.3 OLAP Operations (EXAM QUESTIONS!)

9.3.1 Roll-up & Drill-down



9.3.2 Slice & Dice



9.3.3 Pivot (Rotate)

Pivot						
Αλλαγή της οπτικής γωνίας του cube - εναλλαγή dimensions στους άξονες.						
Example:						
	Q1	Q2	Pivot →	TV	Radio	
TV	100	150		Q1	100	50
Radio	50	75		Q2	150	75

9.4 Solved Exam Problems

9.4.1 2024-25 Exam Questions 1-3 Style

OLAP Operations Identification

Sales Data Cube: Dimensions = (Time, Product, Location)

Question 1: View sales by quarter instead of month.

Answer: Roll-up on Time dimension (month → quarter)

Question 2: View sales for Q1 2024 only.

Answer: Slice (selection on Time = 'Q1 2024')

Question 3: View sales for Q1 2024, in Athens, for Electronics.

Answer: Dice (selection on multiple dimensions)

9.4.2 2024-25 Exam Question 15 Style

Data Cube Construction

Given: Fact table with (Product, Store, Time, Sales)

Question: How many cells in a data cube with:

- 10 products
- 5 stores
- 4 quarters

Answer:

- Base cuboid: $10 \times 5 \times 4 = 200$ cells
- With aggregates (ALL):
 - Product $\times$ Store $\times$ Time: 200
 - Product $\times$ Store $\times$ ALL: $10 \times 5 = 50$
 - Product $\times$ ALL $\times$ Time: $10 \times 4 = 40$
 - ALL $\times$ Store $\times$ Time: $5 \times 4 = 20$
 - Product $\times$ ALL $\times$ ALL: 10
 - ALL $\times$ Store $\times$ ALL: 5
 - ALL $\times$ ALL $\times$ Time: 4
 - ALL $\times$ ALL $\times$ ALL: 1
- Total: $200 + 50 + 40 + 20 + 10 + 5 + 4 + 1 = 330$ cells

General Formula: For dimensions with $d_1, d_2, \dots, d_n$ values:

$$\text{Total cells} = (d_1 + 1)(d_2 + 1) \dots (d_n + 1)$$

Here: $(10 + 1)(5 + 1)(4 + 1) = 11 \times 6 \times 5 = 330$

9.5 Common Mistakes

Συχνά Λάθη στις Εξετάσεις

1. Roll-up vs Drill-down confusion:

- Roll-up: Less detail, MORE aggregation (day→month)
- Drill-down: More detail, LESS aggregation (month→day)

2. Slice vs Dice:

- Slice: ONE dimension selection
- Dice: MULTIPLE dimension selection

3. Star vs Snowflake:

- Star: Denormalized dimensions (faster queries)
- Snowflake: Normalized dimensions (less redundancy)

4. Cube cell count:

- Don't forget the ALL aggregations (+1 for each dimension)

9.6 Quick Reference

OLAP Operations Summary

OLAP Operations:

- **Roll-up:** Aggregate (less detail): day→month→year
- **Drill-down:** Disaggregate (more detail): year→month→day
- **Slice:** Filter ONE dimension
- **Dice:** Filter MULTIPLE dimensions
- **Pivot:** Rotate axes

Schemas:

- Star: Fact table + denormalized dimensions (simple, fast)
- Snowflake: Normalized dimensions (complex, less redundant)

Cube Cells:

$(d_1 + 1)(d_2 + 1) \dots (d_n + 1)$
(+1 for ALL aggregation)

Remember:

Roll-up = UP the hierarchy
Drill-down = DOWN to detail

Κεφάλαιο 10

RAG & Retrieval-based LLMs

Key Concepts

RAG (Retrieval-Augmented Generation):

Συνδυασμός retrieval από knowledge base + generation από LLM.

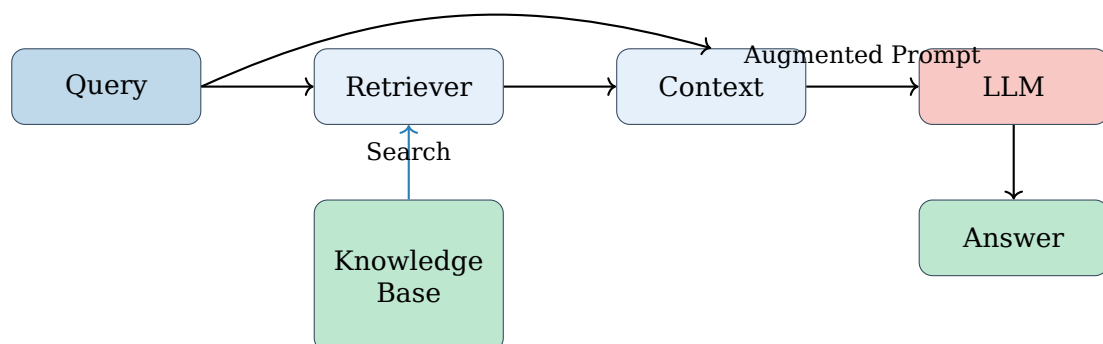
Basic Pipeline:

1. **Query:** User question
2. **Retrieve:** Find relevant documents
3. **Augment:** Add context to prompt
4. **Generate:** LLM produces answer

Key Metrics:

- Recall@K: Fraction of relevant docs in top-K
- Precision@K: Fraction of top-K that are relevant
- MRR (Mean Reciprocal Rank): $\frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$

10.1 RAG Architecture



10.2 Retrieval Methods

10.2.1 Sparse Retrieval (Traditional)

TF-IDF / BM25

Keyword-based retrieval using term frequencies.

TF-IDF:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{\text{DF}(t)} \right)$$

BM25: Improved TF-IDF with document length normalization.

Pros: Fast, interpretable, no training required

Cons: No semantic understanding (keyword mismatch problem)

10.2.2 Dense Retrieval (Neural)

Dense Embeddings

Documents and queries encoded as vectors in semantic space.

Process:

1. Encode documents $\rightarrow \mathbf{d}_i = \text{Encoder}(d_i)$
2. Encode query $\rightarrow \mathbf{q} = \text{Encoder}(q)$
3. Score = $\cos(\mathbf{q}, \mathbf{d}_i)$ or $\mathbf{q} \cdot \mathbf{d}_i$

Pros: Captures semantic similarity

Cons: Requires training, less interpretable

Sparse (TF-IDF/BM25)

Keyword matching
Exact terms
Bag-of-words

vs

Dense (Embeddings)

Semantic matching
Vector similarity
Context-aware

10.3 Vector Databases

Vector Similarity Search

For efficient retrieval from millions of embeddings:

Exact Search:

- Brute-force comparison with all vectors
- $O(n)$ complexity

Approximate Nearest Neighbor (ANN):

- HNSW (Hierarchical Navigable Small World)
- IVF (Inverted File Index)
- Product Quantization (PQ)

Trade-off: Speed vs Accuracy

10.4 Chunking Strategies

Document Chunking

Splitting documents into smaller pieces for retrieval.

Strategies:

- **Fixed-size:** Every N tokens/characters
- **Sentence-based:** Split at sentence boundaries
- **Paragraph-based:** Natural paragraph breaks
- **Semantic:** Use embeddings to find natural breaks
- **Overlapping:** Chunks with shared context

Considerations:

- Too small: Lost context
- Too large: Irrelevant content included
- Typical: 256-512 tokens with 20% overlap

10.5 RAG Evaluation

Metric	Formula	Use Case
Recall@K	$\frac{ \text{relevant} \cap \text{top-K} }{ \text{relevant} }$	Coverage
Precision@K	$\frac{ \text{relevant} \cap \text{top-K} }{K}$	Accuracy
MRR	$\frac{1}{ Q } \sum \frac{1}{\text{rank}_i}$	Ranking quality
NDCG	Discounted cumulative gain	Graded relevance

10.6 Common Challenges

RAG Challenges

- 1. Retrieval Failure:**
 - Relevant documents not retrieved
 - Solution: Better embeddings, hybrid retrieval
- 2. Context Window Limit:**
 - Too much retrieved content
 - Solution: Re-ranking, summarization
- 3. Hallucination:**
 - LLM ignores or contradicts retrieved context
 - Solution: Better prompting, fact verification
- 4. Latency:**
 - Retrieval adds overhead
 - Solution: Caching, ANN indexes

10.7 Advanced RAG Techniques

RAG Improvements

Hybrid Retrieval: Combine sparse (BM25) + dense retrieval, then merge results.
Re-ranking: Use cross-encoder to re-score top-K candidates.
Query Expansion: Use LLM to generate alternative queries.
Multi-hop RAG: Iterative retrieval for complex questions.

10.8 Quick Reference

RAG Summary

RAG Pipeline: Query → Retrieve → Augment → Generate

Retrieval Types:

- Sparse (BM25): Keyword matching, fast, no training
- Dense (Embeddings): Semantic matching, requires training

Key Design Choices:

- Chunk size (256-512 tokens typical)
- Embedding model selection
- Number of retrieved documents (K)

Why RAG?

Up-to-date knowledge
Reduces hallucination

Hybrid = Best:

BM25 + Dense retrieval
then Re-rank

Κεφάλαιο 11

Formulas Cheatsheet

Συγκεντρωτικός οδηγός όλων των τύπων για γρήγορη αναφορά στις εξετάσεις

11.1 Relational Algebra

Relational Algebra Operators

$\sigma_{condition}(R)$	Selection	Rows	WHERE
$\pi_{cols}(R)$	Projection	Columns	SELECT
$R \cup S$	Union	Both	UNION
$R \cap S$	Intersection	Common	INTERSECT
$R - S$	Difference	Only in R	EXCEPT
$R \bowtie S$	Natural Join	Common cols	NATURAL JOIN
$R \bowtie_{\theta} S$	Theta Join	Condition	JOIN ON

11.2 SQL Essentials

SQL Execution Order

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

WHERE: before grouping | HAVING: after grouping (with aggregates)

11.3 Association Rules

Support

$$\frac{|T \text{ with } X|}{|T|}$$

Confidence

$$\frac{Sup(X \cup Y)}{Sup(X)}$$

Lift

$$\frac{Conf(X \rightarrow Y)}{Sup(Y)}$$

Lift Interpretation: = 1: Independent | > 1: Positive | < 1: Negative

11.4 Distance Measures

Euclidean

$$\sqrt{\sum_i (x_i - y_i)^2}$$

Manhattan

$$\sum_i |x_i - y_i|$$

Cosine Similarity

$$\frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

11.5 Classification Metrics

	Pred+	Pred-
Act+	TP	FN
Act-	FP	TN

$$\text{Acc} = \frac{TP+TN}{\text{All}}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$\text{Spec} = \frac{TN}{TN+FP}$$

$$\text{F1} = \frac{2 \cdot P \cdot R}{P+R}$$

$$\text{FPR} = 1 - \text{Spec}$$

11.6 Decision Trees

Entropy

$$H(S) = - \sum_i p_i \log_2 p_i$$

Μέτρο ακαθαρσίας

Information Gain

$$IG(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

Μείωση entropy με split

Gini Index

$$\text{Gini}(S) = 1 - \sum_i p_i^2$$

Alternative to entropy

Goal: Maximize Information Gain (or minimize Gini) at each split

11.7 HMM - Viterbi Algorithm

Viterbi Recurrence

$$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$$

Init: $\delta_1(i) = \pi_i \cdot b_i(o_1)$ | Backtrack using $\psi_t(j) = \arg \max_i [\delta_{t-1}(i) \cdot a_{ij}]$

11.8 Bloom Filter

False Positive Rate

$$p \approx (1 - e^{-kn/m})^k$$

Optimal Hash Functions

$$k = \frac{m}{n} \ln 2 \approx 0.693 \frac{m}{n}$$

Key: No false negatives! "No" = definitely not in set | "Yes" = maybe in set

11.9 OLAP Operations

Roll-up	Aggregate (day→month→year)
Drill-down	Detail (year→month→day)
Slice	Filter ONE dimension
Dice	Filter MULTIPLE dimensions
Pivot	Rotate axes

Cube Cells: $(d_1 + 1)(d_2 + 1) \dots (d_n + 1)$ including ALL aggregates

11.10 Data Preprocessing

Attribute Types (NOIR)

Nominal ($=$, $\neq$) → **Ordinal** ($<$, $>$) → **Interval** ($+$, $-$) → **Ratio** ($\times$, $\div$)

Test: Can say "twice as much"? Yes → Ratio | No → Interval

Equal-Width:

Equal-Frequency:

Min-Max Normalization:

Z-Score:

$$\text{Width} = \frac{\max - \min}{k}$$

$$\text{Records per bin} = \frac{n}{k}$$

$$x' = \frac{x - \min}{\max - \min}$$

$$z = \frac{x - \mu}{\sigma}$$

11.11 Query Processing

Join Algorithm	Best When	Cost
Nested Loop	Small table / Index	$O(n \times m)$
Hash Join	Equality + Memory	$O(n + m)$
Merge Join	Sorted data	$O(n + m)$

Pipelined: σ , π , Nested Loop outer | **Blocking:** Sort, Hash build, Aggregation

11.12 Quick Reference Table

Topic	Key Formula/Concept	Notes
ER $\rightarrow$ Relational	M:N $\rightarrow$ New table	1:N $\rightarrow$ FK in N-side
Selection (σ)	Filter rows	= WHERE
Projection (π)	Choose columns	= SELECT (removes dups)
Natural Join ($\bowtie$)	Match ALL common cols	Auto-match
Support	$ T \text{ with } X / T $	Frequency
Confidence	$Sup(X \cup Y)/Sup(X)$	Conditional
Lift	> 1 : positive correlation	
Precision	$TP/(TP + FP)$	Predicted +
Recall	$TP/(TP + FN)$	Actual +
F1	Harmonic mean P&R	
Viterbi	$\max[\delta \cdot a] \cdot b$	Don't forget b !
Bloom Filter	No false negatives	Only false positives
Roll-up	Less detail	day $\rightarrow$ month
Drill-down	More detail	month $\rightarrow$ day

Κεφάλαιο 12

Συλλογή Λυμένων Ασκήσεων

Αυτό το κεφάλαιο περιέχει μια ολοκληρωμένη συλλογή λυμένων ασκήσεων που καλύπτουν όλη την ύλη του μαθήματος Data Mining. Κάθε άσκηση συνοδεύεται από αναλυτική επεξήγηση βήμα-βήμα.

12.1 E-R Μοντέλο και Σχεδιασμός Βάσεων Δεδομένων

12.1.1 Άσκηση 1.1: Σχεδιασμός E-R από Απαιτήσεις

Εκφώνηση

Σχεδιάστε ένα E-R διάγραμμα για ένα σύστημα music streaming με τις εξής απαιτήσεις:

- Κάθε **τραγούδι** έχει μοναδικό ID, τίτλο, διάρκεια και έτος κυκλοφορίας
- Κάθε **καλλιτέχνης** έχει μοναδικό ID, όνομα και χώρα προέλευσης
- Κάθε τραγούδι ανήκει σε ακριβώς έναν καλλιτέχνη, αλλά ένας καλλιτέχνης μπορεί να έχει πολλά τραγούδια
- Οι **χρήστες** έχουν ID, username και email
- Οι χρήστες δημιουργούν **playlists** με όνομα και ημερομηνία δημιουργίας
- Μια playlist μπορεί να περιέχει πολλά τραγούδια και ένα τραγούδι μπορεί να είναι σε πολλές playlists

Λύση

Βήμα 1: Αναγνώριση Οντοτήτων

Γιατί: Πρώτα εντοπίζουμε τα «αντικείμενα» του συστήματος που έχουν ανεξάρτητη ύπαρξη.

Οντότητες:

- Song (τραγούδι)
- Artist (καλλιτέχνης)
- User (χρήστης)
- Playlist

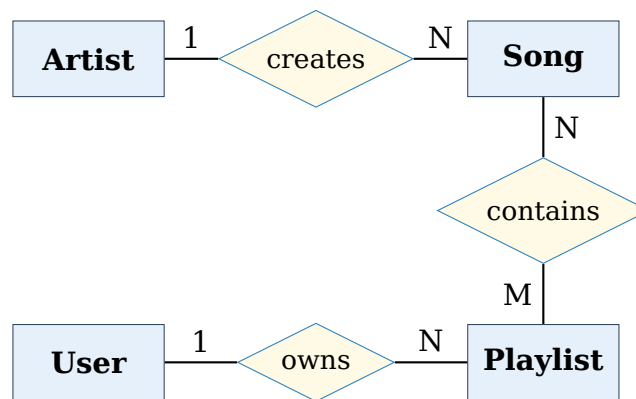
Βήμα 2: Αναγνώριση Γνωρισμάτων

Οντότητα	Γνωρίσματα
Song	<u>song_id</u> , title, duration, release_year
Artist	<u>artist_id</u> , name, country
User	<u>user_id</u> , username, email
Playlist	<u>playlist_id</u> , name, created_date

Βήμα 3: Αναγνώριση Συσχετίσεων και Cardinality

- Artist — creates — Song: **1:N**
- User — owns — Playlist: **1:N**
- Playlist — contains — Song: **M:N**

Βήμα 4: E-R Διάγραμμα



Συνηθές Λάθος

Πολλοί φοιτητές ξεχνούν ότι η σχέση contains είναι M:N και χρειάζεται ενδιάμεσο πίνακα στο σχεσιακό μοντέλο!

Τελική Απάντηση: Το E-R διάγραμμα περιλαμβάνει 4 οντότητες και 3 συσχετίσεις (1:N, 1:N, M:N).

12.1.2 Άσκηση 1.2: Μετατροπή E-R σε Σχεσιακό (M:N)

Εκφώνηση

Μετατρέψτε την M:N σχέση contains μεταξύ Playlist και Song σε σχεσιακούς πίνακες. Η σχέση έχει επιπλέον γνώρισμα position που δηλώνει τη θέση του τραγουδιού στη playlist.

Λύση

Βήμα 1: Κανόνας Μετατροπής M:N

Γιατί: Οι M:N σχέσεις δεν μπορούν να αναπαρασταθούν με ξένα κλειδιά σε έναν από τους δύο πίνακες. Χρειάζεται ενδιάμεσος πίνακας.

Βήμα 2: Δημιουργία Πινάκων


```

1  -- Πίνακας Playlist
2  CREATE TABLE Playlist (
3      playlist_id INT PRIMARY KEY,
4      name VARCHAR(100),
5      created_date DATE,
6      user_id INT REFERENCES User(user_id)
7  );
8
9  -- Πίνακας Song
10 CREATE TABLE Song (
11     song_id INT PRIMARY KEY,
12     title VARCHAR(200),
13     duration INT,
14     release_year INT,
15     artist_id INT REFERENCES Artist(artist_id)
16 );
17
18 -- Ενδιάμεσος πίνακας για M:N
19 CREATE TABLE Playlist_Song (
20     playlist_id INT REFERENCES Playlist(playlist_id),
21     song_id INT REFERENCES Song(song_id),
22     position INT, -- Γνώρισμα της σχέσης
23     PRIMARY KEY (playlist_id, song_id)
24 );

```

Σημαντικό

Το πρωτεύον κλειδί του ενδιάμεσου πίνακα είναι το συνδυασμένο κλειδί (playlist_id, song_id) που αποτρέπει διπλότυπες εγγραφές.

Τελική Απάντηση: Δημιουργούνται 3 πίνακες: Playlist, Song, και Playlist_Song (ενδιάμεσος).

12.1.3 Άσκηση 1.3: Weak Entity Μετατροπή

Εκφώνηση

Ένα σύστημα διαχείρισης παραγγελιών έχει:

- Order με order_id και order_date
- OrderItem (weak entity) με item_number, quantity, price

Το OrderItem εξαρτάται από το Order για την αναγνώρισή του. Μετατρέψτε σε σχεσιακό μοντέλο.

Λύση

Βήμα 1: Αναγνώριση Weak Entity

Γιατί: Μια weak entity δεν έχει αρκετά γνωρίσματα για να ταυτοποιηθεί μόνη της. Το item_number (π.χ. 1, 2, 3) επαναλαμβάνεται μεταξύ διαφορετικών παραγγελιών.

Βήμα 2: Σύνθετο Πρωτεύον Κλειδί

Το πρωτεύον κλειδί της weak entity σχηματίζεται από:

- Το πρωτεύον κλειδί της «ιδιοκτήτριας» οντότητας (owner entity)
- Το μερικό κλειδί (partial key) της weak entity

Βήμα 3: Σχεσιακοί Πίνακες

```

1 CREATE TABLE Order (
2     order_id INT PRIMARY KEY,
3     order_date DATE
4 );
5
6 CREATE TABLE OrderItem (
7     order_id INT REFERENCES Order(order_id),
8     item_number INT,
9     quantity INT,
10    price DECIMAL(10,2),
11    PRIMARY KEY (order_id, item_number) -- Σύνθετοκλειδί
12 );

```

Συνήθες Λάθος

Μην ξεχνάτε: το ξένο κλειδί (order_id) γίνεται **μέρος** του πρωτεύοντος κλειδιού στη weak entity, όχι απλά ξένο κλειδί!

Τελική Απάντηση: PK του OrderItem = (order_id, item_number)

12.1.4 Άσκηση 1.4: Cardinality Constraints

Εκφώνηση

Για τις παρακάτω περιγραφές, προσδιορίστε το cardinality (1:1, 1:N, M:N):

1. Ένας φοιτητής εγγράφεται σε πολλά μαθήματα, ένα μάθημα έχει πολλούς φοιτητές
2. Κάθε υπάλληλος έχει ακριβώς έναν διευθυντή, ένας διευθυντής διευθύνει πολλούς υπαλλήλους
3. Κάθε χώρα έχει μία πρωτεύουσα, κάθε πρωτεύουσα ανήκει σε μία χώρα
4. Ένας συγγραφέας γράφει πολλά βιβλία, ένα βιβλίο μπορεί να έχει πολλούς συγγραφείς

Λύση

Μέθοδος: Για κάθε περίπτωση ρωτάμε:

- Πόσες οντότητες B συνδέονται με μία οντότητα A;
- Πόσες οντότητες A συνδέονται με μία οντότητα B;

#	Σχέση	Cardinality	Εξήγηση
1	Student-Course	M:N	Πολλοί-σε-πολλούς
2	Employee-Manager	N:1	Πολλοί υπάλληλοι, ένας διευθυντής
3	Country-Capital	1:1	Ένα-προς-ένα
4	Author-Book	M:N	Πολλοί-σε-πολλούς

Υλοποίηση σε Σχεσιακό

- **1:1**: FK σε οποιονδήποτε πίνακα (+ UNIQUE)
- **1:N**: FK στην πλευρά του N
- **M:N**: Ενδιάμεσος πίνακας (junction table)

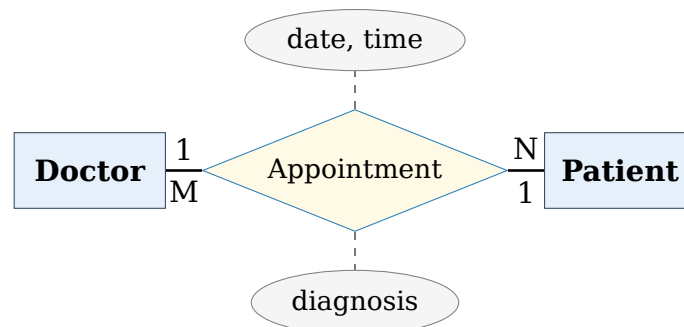
12.1.5 Άσκηση 1.5: Σύνθετο Schema (Hospital System)**Εκφώνηση**

Σχεδιάστε E-R και μετατρέψτε σε σχεσιακό για νοσοκομειακό σύστημα:

- Doctor: doctor_id, name, specialty
- Patient: patient_id, name, birth_date
- Appointment: date, time, diagnosis (σχέση μεταξύ Doctor-Patient)
- Ένας γιατρός έχει πολλά ραντεβού, ένας ασθενής έχει πολλά ραντεβού
- Ένα ραντεβού συνδέει ακριβώς έναν γιατρό με έναν ασθενή

Λύση**Βήμα 1: Ανάλυση**

Η σχέση Doctor-Patient είναι M:N (ένας γιατρός βλέπει πολλούς ασθενείς, ένας ασθενής επισκέπτεται πολλούς γιατρούς), αλλά κάθε συγκεκριμένο ραντεβού αφορά ακριβώς έναν γιατρό και έναν ασθενή.

Βήμα 2: E-R Διάγραμμα**Βήμα 3: Σχεσιακό Μοντέλο**

```

1 CREATE TABLE Doctor (
2   doctor_id INT PRIMARY KEY,
3   name VARCHAR(100),
4   specialty VARCHAR(50)
5 );
6
7 CREATE TABLE Patient (
8   patient_id INT PRIMARY KEY,
9   name VARCHAR(100),
10  birth_date DATE
11 );
12
13 CREATE TABLE Appointment (
  
```

```

14 appointment_id INT PRIMARY KEY,
15 doctor_id INT REFERENCES Doctor(doctor_id),
16 patient_id INT REFERENCES Patient(patient_id),
17 appointment_date DATE,
18 appointment_time TIME,
19 diagnosis TEXT
20 );

```

Τελική Απάντηση: 3 πίνακες με τον Appointment να περιέχει FKs και προς τους δύο.

12.2 Σχεσιακή Άλγεβρα

Σύνοψη Τελεστών

Σύμβολο	Όνομα	Περιγραφή
σ	Selection	Επιλογή γραμμών
π	Projection	Επιλογή στηλών
$\cup$	Union	Ένωση συνόλων
$\cap$	Intersection	Τομή συνόλων
$-$	Difference	Διαφορά συνόλων
$\times$	Cartesian Product	Καρτεσιανό γινόμενο
$\bowtie$	Natural Join	Φυσική σύζευξη
$\bowtie_{\theta}$	Theta Join	Σύζευξη με συνθήκη
$\div$	Division	Διαίρεση

12.2.1 Άσκηση 2.1: Selection και Projection

Εκφώνηση

Δίνεται ο πίνακας Employee(emp_id, name, department, salary).
Γράψτε σε σχεσιακή άλγεβρα: «Βρες τα ονόματα των υπαλλήλων του τμήματος IT με μισθό πάνω από 50000».

Λύση

Βήμα 1: Selection (επιλογή γραμμών)

Γιατί: Πρώτα φιλτράρουμε τις γραμμές που πληρούν τις συνθήκες.

$$\sigma_{department='IT' \wedge salary > 50000}(Employee)$$

Βήμα 2: Projection (επιλογή στηλών)

Γιατί: Μετά κρατάμε μόνο τη στήλη που μας ενδιαφέρει.

$$\pi_{name}(\sigma_{department='IT' \wedge salary > 50000}(Employee))$$

Σειρά Εφαρμογής

Η σειρά Selection → Projection είναι σημαντική για βελτιστοποίηση! Αν κά-
νουμε πρώτα projection, χάνουμε τις στήλες που χρειαζόμαστε για το φιλ-
τράρισμα.

Τελική Απάντηση: $\pi_{name}(\sigma_{department='IT' \wedge salary > 50000}(Employee))$

12.2.2 Άσκηση 2.2: Natural Join**Εκφώνηση**

Δίνονται:

- Student(sid, sname, dept)
- Enrollment(sid, cid, grade)
- Course(cid, cname, credits)

Βρείτε τα ονόματα φοιτητών και τα μαθήματα στα οποία είναι εγγεγραμμέ-
νοι.

Λύση**Βήμα 1: Αναγνώριση κοινών γνωρισμάτων**

- Student ⋈ Enrollment: κοινό sid
- Enrollment ⋈ Course: κοινό cid

Βήμα 2: Natural Join αλυσίδα

$Student \bowtie Enrollment \bowtie Course$

Βήμα 3: Projection

$\pi_{sname, cname}(Student \bowtie Enrollment \bowtie Course)$

Πώς λειτουργεί το Natural Join

Το ⋈ συνδέει πλειάδες που έχουν **ίδιες τιμές** στα κοινά γνωρίσματα (αυτά με το ίδιο όνομα). Οι κοινές στήλες εμφανίζονται μία φορά στο αποτέλεσμα.

12.2.3 Άσκηση 2.3: Theta Join**Εκφώνηση**

Δίνονται:

- Employee(eid, ename, salary, manager_id)
- Manager(mid, mname, bonus)

Βρείτε ζεύγη (υπάλληλος, διευθυντής) όπου ο μισθός του υπαλλήλου είναι
μεγαλύτερος από το bonus του διευθυντή του.

Λύση

Βήμα 1: Σύνδεση υπαλλήλου με διευθυντή

Πρώτα συνδέουμε με βάση το $\text{manager_id} = \text{mid}$:

$$\text{Employee} \bowtie_{\text{manager_id}=\text{mid}} \text{Manager}$$

Βήμα 2: Φιλτράρισμα

Εφαρμόζουμε τη συνθήκη $\text{salary} > \text{bonus}$:

$$\sigma_{\text{salary}>\text{bonus}}(\text{Employee} \bowtie_{\text{manager_id}=\text{mid}} \text{Manager})$$

Βήμα 3: Projection

$$\pi_{\text{ename},\text{mname}}(\sigma_{\text{salary}>\text{bonus}}(\text{Employee} \bowtie_{\text{manager_id}=\text{mid}} \text{Manager}))$$

Εναλλακτικά (ενιαίο Theta Join):

$$\pi_{\text{ename},\text{mname}}(\text{Employee} \bowtie_{\text{manager_id}=\text{mid} \wedge \text{salary}>\text{bonus}} \text{Manager})$$

12.2.4 Άσκηση 2.4: Set Difference

Εκφώνηση

Δίνονται:

- AllStudents(sid, sname)
- Enrollment(sid, cid)

Βρείτε τους φοιτητές που **δεν** είναι εγγεγραμμένοι σε κανένα μάθημα.

Λύση

Βήμα 1: Εύρεση εγγεγραμμένων φοιτητών

$$\text{EnrolledStudents} = \pi_{\text{sid}}(\text{Enrollment})$$

Βήμα 2: Set Difference

Γιατί: Το $A - B$ δίνει τα στοιχεία που είναι στο A αλλά όχι στο B.

$$\text{NotEnrolled} = \pi_{\text{sid}}(\text{AllStudents}) - \pi_{\text{sid}}(\text{Enrollment})$$

Βήμα 3: Επαναφορά ονομάτων (αν χρειάζεται)

$$\pi_{\text{sname}}(\text{AllStudents} \bowtie \text{NotEnrolled})$$

Απαίτηση Συμβατότητας

Για τελεστές συνόλων ($\cup$, $\cap$, $-$), οι σχέσεις πρέπει να έχουν:

1. Ίδιο αριθμό γνωρισμάτων
2. Συμβατά domains στις αντίστοιχες θέσεις

12.2.5 Άσκηση 2.5: Intersection με Projection

Εκφώνηση

Δίνονται:

- $CS_Students(sid, sname)$ – Φοιτητές πληροφορικής
- $Math_Students(sid, sname)$ – Φοιτητές μαθηματικών

Βρείτε τους φοιτητές που σπουδάζουν **και** πληροφορική **και** μαθηματικά.

Λύση

$$CS_Students \cap Math_Students$$

ή ισοδύναμα (αν θέλουμε μόνο ονόματα):

$$\pi_{sname}(CS_Students \cap Math_Students)$$

Ισοδυναμία

Η τομή μπορεί να εκφραστεί με διαφορά:

$$A \cap B = A - (A - B)$$

12.2.6 Άσκηση 2.6: Division (Απλή)**Εκφώνηση**

Δίνονται:

- $Takes(sid, cid)$ – φοιτητής παίρνει μάθημα
- $Required(cid)$ – υποχρεωτικά μαθήματα: $\{C1, C2\}$

Βρείτε τους φοιτητές που έχουν πάρει **όλα** τα υποχρεωτικά μαθήματα.

Λύση

Η Division ($\div$) απαντά σε ερωτήματα «για όλα»:

$$Takes \div Required$$

Βήμα-βήμα αλγόριθμος:

1. Βρες όλους τους φοιτητές: $\pi_{sid}(Takes)$
2. Βρες τι «θα έπρεπε» να έχει πάρει κάθε φοιτητής: $\pi_{sid}(Takes) \times Required$
3. Βρες τι «δεν έχει» πάρει: $(\pi_{sid}(Takes) \times Required) - Takes$
4. Αφαίρεσε όσους «δεν έχουν» κάτι: $\pi_{sid}(Takes) - \pi_{sid}((\pi_{sid}(Takes) \times Required) - Takes)$

Παράδειγμα:

Takes		Required	Result
sid	cid		
S1	C1	cid	sid
S1	C2		
S1	C3	C1	S1
S2	C1	C2	S3
S3	C1		
S3	C2		

12.2.7 Άσκηση 2.7: Division (Σύνθετη)

Εκφώνηση

Δίνονται:

- Supplies(supplier, part)
- Project(pname, part) – έργα και τα parts που χρειάζονται

Βρείτε τους προμηθευτές που μπορούν να προμηθεύσουν **όλα** τα parts που χρειάζεται το έργο «Rocket».

Λύση

Βήμα 1: Βρες τα parts του Rocket

$$RocketParts = \pi_{part}(\sigma_{pname='Rocket'}(Project))$$

Βήμα 2: Division

$$Supplies \div RocketParts$$

Σε SQL (για κατανόηση):

```

1 SELECT DISTINCT s1.supplier
2 FROM Supplies s1
3 WHERE NOT EXISTS (
4     SELECT part FROM Project WHERE pname = 'Rocket'
5     EXCEPT
6     SELECT part FROM Supplies s2
7     WHERE s2.supplier = s1.supplier
8 );

```

12.2.8 Άσκηση 2.8: Σύνθετη Έκφραση

Εκφώνηση

Δίνονται:

- Employee(eid, ename, dept_id, salary)
- Department(dept_id, dname, budget)
- Project(pid, pname, dept_id)

- WorksOn(eid, pid, hours)

Βρείτε τα ονόματα υπαλλήλων του τμήματος «Research» που εργάζονται πάνω από 20 ώρες σε κάποιο project.

Λύση

Βήμα 1: Εύρεση dept_id του Research

$$ResearchDept = \sigma_{dname='Research'}(Department)$$

Βήμα 2: Υπάλληλοι του Research

$$ResearchEmps = Employee \bowtie ResearchDept$$

Βήμα 3: Υπάλληλοι με >20 ώρες

$$HighHours = \sigma_{hours>20}(WorksOn)$$

Βήμα 4: Συνδυασμός

$$Result = \pi_{ename}(ResearchEmps \bowtie HighHours)$$

Ενιαία έκφραση:

$$\pi_{ename}(\sigma_{dname='Research'}(Department) \bowtie Employee \bowtie \sigma_{hours>20}(WorksOn))$$

12.3 SQL

12.3.1 Άσκηση 3.1: SELECT με WHERE

Εκφώνηση

Δίνεται ο πίνακας Products(pid, pname, category, price, stock). Γράψτε SQL query για να βρείτε τα προϊόντα κατηγορίας «Electronics» με τιμή μεταξύ 100 και 500 ευρώ, ταξινομημένα κατά τιμή (φθίνουσα).

Λύση

```

1 SELECT pid, pname, price
2 FROM Products
3 WHERE category = 'Electronics'
4    AND price BETWEEN 100 AND 500
5 ORDER BY price DESC;
```

BETWEEN

Το BETWEEN 100 AND 500 είναι ισοδύναμο με price >= 100 AND price <= 500 (περιλαμβάνει τα όρια).

12.3.2 Άσκηση 3.2: JOINS (INNER, LEFT)

Εκφώνηση

Δίνονται:

- Customers(cid, cname, city)
- Orders(oid, cid, order_date, total)

1. Βρείτε όλους τους πελάτες με τις παραγγελίες τους
2. Βρείτε όλους τους πελάτες, ακόμα κι αν δεν έχουν παραγγελίες

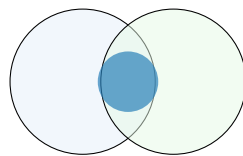
Λύση

(a) INNER JOIN - μόνο πελάτες με παραγγελίες:

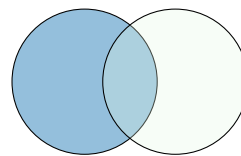
```
1 SELECT c.cname, o.oid, o.total
2 FROM Customers c
3 INNER JOIN Orders o ON c.cid = o.cid;
```

(b) LEFT JOIN - όλοι οι πελάτες:

```
1 SELECT c.cname, o.oid, o.total
2 FROM Customers c
3 LEFT JOIN Orders o ON c.cid = o.cid;
```



INNER JOIN



LEFT JOIN

12.3.3 Άσκηση 3.3: GROUP BY με COUNT

Εκφώνηση

Χρησιμοποιώντας τους πίνακες Orders και Customers, βρείτε πόσες παραγγελίες έχει κάνει κάθε πελάτης.

Λύση

```
1 SELECT c.cid, c.cname, COUNT(o.oid) AS order_count
2 FROM Customers c
3 LEFT JOIN Orders o ON c.cid = o.cid
4 GROUP BY c.cid, c.cname;
```

LEFT vs INNER JOIN

Χρησιμοποιούμε LEFT JOIN για να συμπεριλάβουμε πελάτες με 0 παραγγελίες. Με INNER JOIN θα χάναμε αυτούς τους πελάτες!

12.3.4 Άσκηση 3.4: GROUP BY με HAVING

Εκφώνηση

Βρείτε τις πόλεις που έχουν περισσότερους από 5 πελάτες και εμφανίστε τον αριθμό πελατών ανά πόλη.

Λύση

```
1 SELECT city, COUNT(*) AS customer_count
2 FROM Customers
3 GROUP BY city
4 HAVING COUNT(*) > 5
5 ORDER BY customer_count DESC;
```

WHERE vs HAVING

- **WHERE:** Φιλτράρει γραμμές **πριν** το grouping
- **HAVING:** Φιλτράρει *groups* **μετά** το grouping

12.3.5 Άσκηση 3.5: Subquery με IN

Εκφώνηση

Βρείτε τους πελάτες που έχουν κάνει παραγγελία με total > 1000.

Λύση

Μέθοδος 1: Subquery με IN

```
1 SELECT cname
2 FROM Customers
3 WHERE cid IN (
4     SELECT cid
5     FROM Orders
6     WHERE total > 1000
7 );
```

Μέθοδος 2: JOIN (ισοδύναμο)

```
1 SELECT DISTINCT c.cname
2 FROM Customers c
3 JOIN Orders o ON c.cid = o.cid
4 WHERE o.total > 1000;
```

12.3.6 Άσκηση 3.6: Correlated Subquery

Εκφώνηση

Βρείτε τους υπαλλήλους που έχουν μισθό μεγαλύτερο από τον μέσο όρο του τμήματός τους.

Λύση

```

1 SELECT e1.ename, e1.salary, e1.dept_id
2 FROM Employee e1
3 WHERE e1.salary > (
4     SELECT AVG(e2.salary)
5     FROM Employee e2
6     WHERE e2.dept_id = e1.dept_id  -- Correlated!
7 );

```

Correlated Subquery

Το subquery εξαρτάται από το εξωτερικό query (αναφέρεται στο e1.dept_id). Εκτελείται μία φορά **για κάθε γραμμή** του εξωτερικού query.

12.3.7 Άσκηση 3.7: NOT EXISTS Pattern

Εκφώνηση

Βρείτε τους πελάτες που **δεν** έχουν κάνει καμία παραγγελία.

Λύση

Μέθοδος 1: NOT EXISTS

```

1 SELECT c.cname
2 FROM Customers c
3 WHERE NOT EXISTS (
4     SELECT 1
5     FROM Orders o
6     WHERE o.cid = c.cid
7 );

```

Μέθοδος 2: LEFT JOIN + IS NULL

```

1 SELECT c.cname
2 FROM Customers c
3 LEFT JOIN Orders o ON c.cid = o.cid
4 WHERE o.oid IS NULL;

```

Μέθοδος 3: NOT IN

```

1 SELECT cname
2 FROM Customers
3 WHERE cid NOT IN (SELECT cid FROM Orders);

```

NOT IN με NULL

Αν το subquery του NOT IN επιστρέψει NULL, το αποτέλεσμα είναι κενό! Προτιμήστε NOT EXISTS για ασφάλεια.

12.3.8 Άσκηση 3.8: SQL Division (Double NOT EXISTS)**Εκφώνηση**

Δίνονται:

- Enrollment(sid, cid)
- Course(cid, cname)

Βρείτε τους φοιτητές που έχουν εγγραφεί σε **όλα** τα μαθήματα.

Λύση

Λογική: «Δεν υπάρχει μάθημα στο οποίο ο φοιτητής να μην έχει εγγραφεί»

```

1 SELECT DISTINCT e1.sid
2 FROM Enrollment e1
3 WHERE NOT EXISTS (
4     -- Μαθήματα που ΔΕΝ έχει πάρει ο      e1.sid
5     SELECT c.cid
6     FROM Course c
7     WHERE NOT EXISTS (
8         -- Έλεγχος αν ο e1.sid έχει πάρει το c.cid
9         SELECT 1
10        FROM Enrollment e2
11        WHERE e2.sid = e1.sid AND e2.cid = c.cid
12    )
13 );

```

Ερμηνεία Double NOT EXISTS

1. Εσωτερικό NOT EXISTS: Βρες μαθήματα που **δεν** έχει πάρει ο φοιτητής
2. Εξωτερικό NOT EXISTS: Κράτα φοιτητές που **δεν** έχουν τέτοια μαθήματα

Δηλαδή: φοιτητές χωρίς «κενά» στο πρόγραμμα.

12.3.9 Άσκηση 3.9: Multiple JOINS με Aggregation**Εκφώνηση**

Δίνονται:

- Orders(oid, cid, order_date)
- OrderItems(oid, pid, quantity, unit_price)
- Products(pid, pname, category)

Βρείτε το συνολικό έσοδο (revenue) ανά κατηγορία προϊόντος.

Λύση

```

1 SELECT
2     p.category,
3     SUM(oi.quantity * oi.unit_price) AS total_revenue
4 FROM Products p
5 JOIN OrderItems oi ON p.pid = oi.pid
6 GROUP BY p.category
7 ORDER BY total_revenue DESC;
```

12.3.10 Άσκηση 3.10: Complex Query

Εκφώνηση

Βρείτε τους top 3 πελάτες (με βάση συνολικές αγορές) που έχουν αγοράσει προϊόντα από τουλάχιστον 2 διαφορετικές κατηγορίες.

Λύση

```

1 SELECT
2     c.cid,
3     c.cname,
4     SUM(oi.quantity * oi.unit_price) AS total_spent,
5     COUNT(DISTINCT p.category) AS categories_bought
6 FROM Customers c
7 JOIN Orders o ON c.cid = o.cid
8 JOIN OrderItems oi ON o.oid = oi.oid
9 JOIN Products p ON oi.pid = p.pid
10 GROUP BY c.cid, c.cname
11 HAVING COUNT(DISTINCT p.category) >= 2
12 ORDER BY total_spent DESC
13 LIMIT 3;
```

Σειρά Εκτέλεσης SQL

1. FROM / JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT

12.4 Query Processing

12.4.1 Άσκηση 4.1: Pipelining vs Materialization

Εκφώνηση

Για το query: $\pi_{name}(\sigma_{salary > 50000}(Employee \bowtie Department))$
Ταξινομήστε τους τελεστές σε pipelined ή materialized και εξηγήστε γιατί.

Λύση

Ορισμοί:

- **Pipelining:** Τα αποτελέσματα περνάνε αμέσως στον επόμενο τελεστή χωρίς αποθήκευση
- **Materialization:** Τα αποτελέσματα αποθηκεύονται προσωρινά πριν περάσουν στον επόμενο τελεστή

Τελεστής	Τύπος	Λόγος
$\bowtie$ (Join)	Materialized	Χρειάζεται πλήρη δεδομένα για hash/sort
σ (Selection)	Pipelined	Φιλτράρει γραμμή-γραμμή
π (Projection)	Pipelined	Επεξεργάζεται γραμμή-γραμμή

Pipeline Breakers

Οι τελεστές που **δεν** μπορούν να είναι pipelined ονομάζονται «pipeline breakers»:

- Sort
- Hash Join (build phase)
- Group By / Aggregation
- Duplicate elimination (DISTINCT)

12.4.2 Άσκηση 4.2: Επιλογή Join Algorithm

Εκφώνηση

Δίνονται δύο πίνακες:

- R : 10,000 tuples, 100 tuples/page $\Rightarrow$ 100 pages
- S : 1,000 tuples, 100 tuples/page $\Rightarrow$ 10 pages
- Buffer: 12 pages

Ποιος αλγόριθμος join είναι καλύτερος; Υπολογίστε το I/O cost.

Λύση

Nested Loop Join:

$$Cost_{NLJ} = B_R + B_R \cdot B_S = 100 + 100 \cdot 10 = 1100 \text{ I/Os}$$

(με το μικρότερο ως outer: $10 + 10 \cdot 100 = 1010$ I/Os)

Block Nested Loop Join:

Με buffer $B = 12$ pages, κρατάμε $B - 2 = 10$ pages για τον outer.

$$Cost_{BNLJ} = B_R + \lceil B_R / (B - 2) \rceil \cdot B_S$$

Με S ως outer (μικρότερο):

$$Cost = 10 + \lceil 10/10 \rceil \cdot 100 = 10 + 100 = 110 \text{ I/Os}$$

Hash Join:

$$Cost_{Hash} = 3 \cdot (B_R + B_S) = 3 \cdot 110 = 330 \text{ I/Os}$$

Sort-Merge Join:

$$Cost_{SM} = Sort(R) + Sort(S) + B_R + B_S$$

$$= 2 \cdot 100 \cdot \lceil \log_{11}(10) \rceil + 2 \cdot 10 \cdot \lceil \log_{11}(1) \rceil + 110$$

$$\approx 200 \cdot 1 + 20 \cdot 1 + 110 = 330 \text{ I/Os}$$

Algorithm	I/O Cost
Simple NLJ (R outer)	1,100
Simple NLJ (S outer)	1,010
Block NLJ (S outer)	110
Hash Join	330
Sort-Merge	~330

Τελική Απάντηση: Block Nested Loop Join με S ως outer (110 I/Os).

12.4.3 Άσκηση 4.3: Query Optimization (Push Selection)

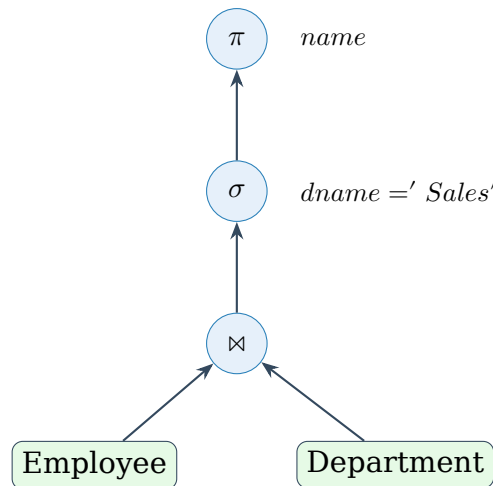
Εκφώνηση

Βελτιστοποιήστε το query tree για:

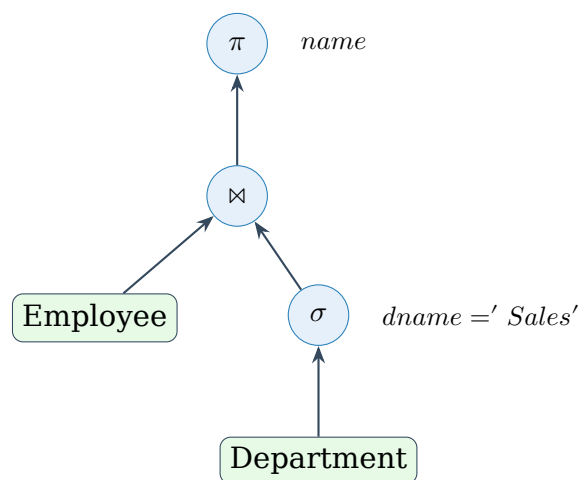
$$\pi_{ename}(\sigma_{dname='Sales'}(Employee \bowtie Department))$$

Λύση

Αρχικό Query Tree:



Βελτιστοποιημένο Query Tree (Push Selection Down):



Κανόνες Βελτιστοποίησης

1. **Push selections down:** Μειώνει το μέγεθος ενδιάμεσων αποτελεσμάτων
2. **Push projections down:** Μειώνει το πλάτος tuples
3. **Join ordering:** Μικρότερα ενδιάμεσα αποτελέσματα πρώτα

12.4.4 Άσκηση 4.4: Cost Comparison

Εκφώνηση

Για τον πίνακα R με 10,000 tuples και index στο attribute A :
 Συγκρίνετε το cost για: $\sigma_{A=5}(R)$

1. Table scan
2. B+-tree index (height = 3, clustering factor = 1.0)
3. Hash index

Λύση

Έστω $B_R = 100$ pages, selectivity = 1% (100 tuples match).

(a) Table Scan:

$$Cost = B_R = 100 \text{ page I/Os}$$

(b) B+-tree Index (clustered):

$$Cost = height + \text{matching pages} = 3 + 1 = 4 \text{ I/Os}$$

(Clustered = τα matching tuples είναι συνεχόμενα)

(c) Hash Index:

$$Cost = 1.2 \text{ (average bucket access)} \approx 2 \text{ I/Os}$$

Method	I/O Cost
Table Scan	100
B+-tree (clustered)	4
Hash Index	~ 2

Unclustered Index

Αν ο B+-tree index ήταν unclustered, το cost θα ήταν:

$$Cost = 3 + 100 = 103 \text{ I/Os (χειρότερο από scan!)}$$

12.5 Προεπεξεργασία Δεδομένων

12.5.1 Άσκηση 5.1: Ταξινόμηση Attribute Types (NOIR)

Εκφώνηση

Ταξινομήστε τα παρακάτω attributes σε Nominal, Ordinal, Interval, ή Ratio:

1. Αριθμός τηλεφώνου
2. Θερμοκρασία σε Celsius
3. Εκπαιδευτικό επίπεδο (Λύκειο, Πτυχίο, Μεταπτυχιακό, PhD)
4. Ηλικία σε έτη
5. Χρώμα ματιών
6. Ημερομηνία γέννησης
7. Βαθμολογία 1-5 αστέρια

Λύση

Attribute	Type	Εξήγηση
Αρ. τηλεφώνου	Nominal	Αναγνωριστικό, δεν έχει νόημα η σειρά/αριθμητικές πράξεις
Θερμοκρασία (°C)	Interval	Διαφορές έχουν νόημα, αλλά 0°C δεν = «καμία θερμοκρασία»
Εκπαιδευτικό επίπεδο	Ordinal	Σειρά έχει νόημα, αλλά διαφορές όχι
Ηλικία	Ratio	Υπάρχει απόλυτο 0, λόγοι έχουν νόημα (διπλάσια ηλικία)
Χρώμα ματιών	Nominal	Κατηγορικό, χωρίς σειρά
Ημερομηνία γέννησης	Interval	Διαφορές έχουν νόημα, δεν υπάρχει «0 ημερομηνία»
Rating 1-5	Ordinal	Σειρά έχει νόημα, αλλά 4 < 2×2

NOIR Summary

- **Nominal:** Κατηγορίες, mode μόνο
- **Ordinal:** + Σειρά, median
- **Interval:** + Διαφορές, mean
- **Ratio:** + Λόγοι, geometric mean

12.5.2 Άσκηση 5.2: Equal-Width Binning**Εκφώνηση**

Δίνονται οι τιμές: {4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34}
Εφαρμόστε equal-width binning με 3 bins.

Λύση**Βήμα 1: Υπολογισμός εύρους**

$$\text{Range} = \max - \min = 34 - 4 = 30$$

Βήμα 2: Υπολογισμός πλάτους bin

$$\text{Width} = \frac{\text{Range}}{\text{num_bins}} = \frac{30}{3} = 10$$

Βήμα 3: Καθορισμός ορίων

- Bin 1: [4, 14)
- Bin 2: [14, 24)
- Bin 3: [24, 34]

Βήμα 4: Κατανομή τιμών

Bin	Τιμές
Bin 1 [4, 14)	4, 8, 9
Bin 2 [14, 24)	15, 21, 21
Bin 3 [24, 34]	24, 25, 26, 28, 29, 34

Μειονέκτημα Equal-Width

Τα bins μπορεί να έχουν πολύ άνιση κατανομή (Bin 3 έχει 6 τιμές, Bin 1 έχει 3).

12.5.3 Άσκηση 5.3: Equal-Frequency Binning

Εκφώνηση

Για τις ίδιες τιμές: {4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34}
Εφαρμόστε equal-frequency (equal-depth) binning με 3 bins.

Λύση

Βήμα 1: Υπολογισμός μεγέθους bin

$$\text{Bin size} = \frac{n}{\text{num_bins}} = \frac{12}{3} = 4 \text{ τιμές/bin}$$

Βήμα 2: Κατανομή (sorted)

Bin	Τιμές
Bin 1	4, 8, 9, 15
Bin 2	21, 21, 24, 25
Bin 3	26, 28, 29, 34

Equal-Frequency vs Equal-Width

- **Equal-Width:** Ίδιο εύρος ανά bin, διαφορετικό πλήθος
- **Equal-Frequency:** Ίδιο πλήθος ανά bin, διαφορετικό εύρος

12.5.4 Άσκηση 5.4: Smoothing by Bin Means

Εκφώνηση

Χρησιμοποιώντας τα equal-frequency bins από την προηγούμενη άσκηση, εφαρμόστε smoothing by bin means.

Λύση

Βήμα 1: Υπολογισμός μέσου όρου κάθε bin

$$\text{Mean}_1 = \frac{4 + 8 + 9 + 15}{4} = \frac{36}{4} = 9$$

$$\text{Mean}_2 = \frac{21 + 21 + 24 + 25}{4} = \frac{91}{4} = 22.75$$

$$\text{Mean}_3 = \frac{26 + 28 + 29 + 34}{4} = \frac{117}{4} = 29.25$$

Βήμα 2: Αντικατάσταση

Bin	Original	Smoothed
Bin 1	4, 8, 9, 15	9, 9, 9, 9
Bin 2	21, 21, 24, 25	22.75, 22.75, 22.75, 22.75
Bin 3	26, 28, 29, 34	29.25, 29.25, 29.25, 29.25

12.5.5 Άσκηση 5.5: Smoothing by Bin Boundaries**Εκφώνηση**

Για τα ίδια bins, εφαρμόστε smoothing by bin boundaries.

Λύση

Μέθοδος: Κάθε τιμή αντικαθίσταται με το πλησιέστερο όριο του bin της.

Bin 1: [4, 15]

- 4 → 4 (είναι το κάτω όριο)
- 8 → 4 (πιο κοντά στο 4 παρά στο 15)
- 9 → 4 (πιο κοντά στο 4)
- 15 → 15 (είναι το άνω όριο)

Bin 2: [21, 25]

- 21 → 21
- 21 → 21
- 24 → 25 (πιο κοντά στο 25)
- 25 → 25

Bin 3: [26, 34]

- 26 → 26
- 28 → 26 (πιο κοντά στο 26)
- 29 → 26 (ή 34, ίση απόσταση - επιλέγουμε το κοντινότερο)
- 34 → 34

Bin	Original	Smoothed
Bin 1	4, 8, 9, 15	4, 4, 4, 15
Bin 2	21, 21, 24, 25	21, 21, 25, 25
Bin 3	26, 28, 29, 34	26, 26, 26, 34

12.5.6 Άσκηση 5.6: Min-Max Normalization

Εκφώνηση

Κανονικοποιήστε τις τιμές $\{10, 20, 30, 40, 50\}$ στο διάστημα $[0, 1]$ χρησιμοποιώντας Min-Max normalization.

Λύση

Τύπος Min-Max Normalization:

$$x' = \frac{x - \min}{\max - \min}$$

Υπολογισμοί:

$\min = 10, \max = 50, \text{range} = 40$

$$10' = \frac{10 - 10}{40} = 0.00$$

$$20' = \frac{20 - 10}{40} = 0.25$$

$$30' = \frac{30 - 10}{40} = 0.50$$

$$40' = \frac{40 - 10}{40} = 0.75$$

$$50' = \frac{50 - 10}{40} = 1.00$$

Γενική Μορφή

Για normalization στο διάστημα $[a, b]$:

$$x' = \frac{x - \min}{\max - \min} \cdot (b - a) + a$$

12.5.7 Άσκηση 5.7: Z-Score Standardization

Εκφώνηση

Για τις ίδιες τιμές $\{10, 20, 30, 40, 50\}$, εφαρμόστε Z-score standardization.

Λύση

Τύπος Z-Score:

$$z = \frac{x - \mu}{\sigma}$$

Βήμα 1: Υπολογισμός μέσου όρου

$$\mu = \frac{10 + 20 + 30 + 40 + 50}{5} = 30$$

Βήμα 2: Υπολογισμός τυπικής απόκλισης

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{n}} = \sqrt{\frac{400 + 100 + 0 + 100 + 400}{5}} = \sqrt{200} \approx 14.14$$

Βήμα 3: Υπολογισμός z-scores

$$\begin{aligned} z_{10} &= \frac{10 - 30}{14.14} \approx -1.41 \\ z_{20} &= \frac{20 - 30}{14.14} \approx -0.71 \\ z_{30} &= \frac{30 - 30}{14.14} = 0.00 \\ z_{40} &= \frac{40 - 30}{14.14} \approx +0.71 \\ z_{50} &= \frac{50 - 30}{14.14} \approx +1.41 \end{aligned}$$

Min-Max vs Z-Score

- **Min-Max:** Bounded $[0, 1]$, ευαίσθητο σε outliers
- **Z-Score:** Unbounded, μέσος=0, std=1, λιγότερο ευαίσθητο σε outliers

12.5.8 Άσκηση 5.8: Υπολογισμός Αποστάσεων**Εκφώνηση**

Δίνονται τα διανύσματα:

- $\mathbf{x} = (1, 2, 3)$
- $\mathbf{y} = (4, 6, 3)$

Υπολογίστε: (a) Euclidean distance, (b) Manhattan distance, (c) Cosine similarity

Λύση**(a) Euclidean Distance:**

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

$$d_E = \sqrt{(1-4)^2 + (2-6)^2 + (3-3)^2} = \sqrt{9 + 16 + 0} = \sqrt{25} = 5$$

(b) Manhattan Distance:

$$d_M(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

$$d_M = |1-4| + |2-6| + |3-3| = 3 + 4 + 0 = 7$$

(c) Cosine Similarity:

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \cdot \|\mathbf{y}\|}$$

$$\mathbf{x} \cdot \mathbf{y} = 1 \cdot 4 + 2 \cdot 6 + 3 \cdot 3 = 4 + 12 + 9 = 25$$

$$\|\mathbf{x}\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$$

$$\|\mathbf{y}\| = \sqrt{4^2 + 6^2 + 3^2} = \sqrt{61}$$

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{25}{\sqrt{14} \cdot \sqrt{61}} = \frac{25}{\sqrt{854}} \approx 0.855$$

Distance Metric	Value
Euclidean	5.00
Manhattan	7
Cosine Similarity	0.855
Cosine Distance	$1 - 0.855 = 0.145$

12.6 Data Streams

12.6.1 Άσκηση 6.1: Bloom Filter Insert & Lookup

Εκφώνηση

Ένα Bloom Filter έχει μέγεθος $m = 10$ bits και χρησιμοποιεί 2 hash functions:

- $h_1(x) = x \bmod 10$
- $h_2(x) = (2x + 3) \bmod 10$

1. Εισάγετε τα στοιχεία: 15, 23, 36
2. Ελέγξτε αν υπάρχουν: 23, 42

Λύση

Βήμα 1: Αρχικοποίηση

Bit array: $[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$

Βήμα 2: Insert 15

$$h_1(15) = 15 \bmod 10 = 5$$

$$h_2(15) = (30 + 3) \bmod 10 = 3$$

Set bits 3, 5: $[0, 0, 0, \mathbf{1}, 0, \mathbf{1}, 0, 0, 0, 0]$

Βήμα 3: Insert 23

$$h_1(23) = 23 \bmod 10 = 3$$

$$h_2(23) = (46 + 3) \bmod 10 = 9$$

Set bits 3, 9: $[0, 0, 0, \mathbf{1}, 0, 1, 0, 0, 0, \mathbf{1}]$

Βήμα 4: Insert 36

$$h_1(36) = 36 \bmod 10 = 6$$

$$h_2(36) = (72 + 3) \bmod 10 = 5$$

Set bits 5, 6: [0, 0, 0, 1, 0, **1**, **1**, 0, 0, 1]

Final bit array: [0, 0, 0, 1, 0, 1, 1, 0, 0, 1]

Lookup 23:

- $h_1(23) = 3 \rightarrow \text{bit}[3] = 1 \checkmark$

- $h_2(23) = 9 \rightarrow \text{bit}[9] = 1 \checkmark$

Αποτέλεσμα: **Probably in set**

Lookup 42:

$$h_1(42) = 42 \bmod 10 = 2$$

$$h_2(42) = (84 + 3) \bmod 10 = 7$$

- $\text{bit}[2] = 0 \times$

Αποτέλεσμα: **Definitely not in set**

12.6.2 Άσκηση 6.2: Bloom Filter False Positive

Εκφώνηση

Με το ίδιο Bloom Filter από την προηγούμενη άσκηση, ελέγξτε αν το 13 είναι false positive.

Λύση

Bit array: [0, 0, 0, 1, 0, 1, 1, 0, 0, 1]

Lookup 13:

$$h_1(13) = 13 \bmod 10 = 3$$

$$h_2(13) = (26 + 3) \bmod 10 = 9$$

- $\text{bit}[3] = 1 \checkmark$

- $\text{bit}[9] = 1 \checkmark$

Αποτέλεσμα: «Probably in set»

Αλλά: Το 13 **δεν** εισήχθη ποτέ! Αυτό είναι **FALSE POSITIVE**.

False Positive Rate

Η πιθανότητα false positive είναι περίπου:

$$P_{fp} \approx (1 - e^{-kn/m})^k$$

όπου k = hash functions, n = inserted elements, m = bits.

Για $k = 2$, $n = 3$, $m = 10$:

$$P_{fp} \approx (1 - e^{-0.6})^2 \approx (0.45)^2 \approx 0.20 = 20\%$$

12.6.3 Άσκηση 6.3: Count-Min Sketch Query

Εκφώνηση

Ένα Count-Min Sketch έχει $d = 2$ hash functions και $w = 5$ counters ανά row. Μετά από updates, ο πίνακας είναι:

	0	1	2	3	4
h_1	3	7	2	5	1
h_2	4	2	8	3	6

Αν $h_1(\text{"apple"}) = 1$ και $h_2(\text{"apple"}) = 3$, ποια είναι η εκτίμηση για το count του "apple"?

Λύση

Count-Min Sketch Query:

$$\hat{f}(x) = \min_{i=1}^d \text{CMS}[i][h_i(x)]$$

Υπολογισμός:

$$\text{CMS}[1][h_1(\text{"apple"})] = \text{CMS}[1][1] = 7$$

$$\text{CMS}[2][h_2(\text{"apple"})] = \text{CMS}[2][3] = 3$$

$$\hat{f}(\text{"apple"}) = \min(7, 3) = 3$$

Γιατί Minimum?

Τα counts μπορούν να αυξηθούν λόγω collisions με άλλα στοιχεία. Το minimum δίνει την καλύτερη (μικρότερη) εκτίμηση, που είναι εγγυημένα $\geq$ actual count.

12.6.4 Άσκηση 6.4: Count-Min Sketch Update

Εκφώνηση

Χρησιμοποιώντας τον ίδιο CMS, εκτελέστε: Update("banana", +2)
Δίνεται: $h_1(\text{"banana"}) = 4$, $h_2(\text{"banana"}) = 2$

Λύση

Update operation:

Για κάθε row i : $\text{CMS}[i][h_i(x)] += c$

Πριν:

	0	1	2	3	4
h_1	3	7	2	5	1
h_2	4	2	8	3	6

Update:

$$\text{CMS}[1][4] = 1 + 2 = 3$$

$$\text{CMS}[2][2] = 8 + 2 = 10$$

Μετά:

	0	1	2	3	4
h_1	3	7	2	5	3
h_2	4	2	10	3	6

12.6.5 Άσκηση 6.5: Error Bounds

Εκφώνηση

Σχεδιάστε Count-Min Sketch με error $\varepsilon = 0.01$ και failure probability $\delta = 0.05$. Υπολογίστε τις διαστάσεις.

Λύση**Τύποι:**

$$w = \lceil e/\varepsilon \rceil \approx \lceil 2.718/0.01 \rceil = 272$$

$$d = \lceil \ln(1/\delta) \rceil = \lceil \ln(20) \rceil = \lceil 3.0 \rceil = 3$$

Εγγύηση:

Με πιθανότητα τουλάχιστον $1 - \delta = 95\%$:

$$\hat{f}(x) \leq f(x) + \varepsilon \cdot N$$

όπου N = συνολικό count όλων των updates.

Απαιτούμενη μνήμη:

$$\text{Memory} = w \times d \times \text{counter_size} = 272 \times 3 \times 4 = 3264 \text{ bytes}$$

12.6.6 Άσκηση 6.6: Bloom Filter vs Count-Min Sketch

Εκφώνηση

Για κάθε σενάριο, επιλέξτε την κατάλληλη δομή (Bloom Filter ή Count-Min Sketch):

1. Έλεγχος αν ένα URL έχει επισκεφθεί πριν
2. Καταμέτρηση επισκέψεων ανά URL
3. Spam filter: έλεγχος αν email είναι σε blacklist
4. Εύρεση heavy hitters σε network traffic

Λύση

#	Δομή	Αιτιολόγηση
1	Bloom Filter	Membership test μόνο (yes/no)
2	Count-Min Sketch	Χρειάζεται καταμέτρηση, όχι μόνο membership
3	Bloom Filter	Membership test σε blacklist
4	Count-Min Sketch	Heavy hitters = υψηλά counts

Σύγκριση

- **Bloom Filter:** Membership queries ($\in?$), no deletions, no counts
- **Count-Min Sketch:** Frequency queries (count?), supports negative updates

12.7 HMM και Διακριτές Ακολουθίες

12.7.1 Άσκηση 7.1: Υποακολουθίες

Εκφώνηση

Δίνεται η ακολουθία $S = \langle A, B, C, D \rangle$.

1. Πόσες υποακολουθίες (subsequences) έχει;
2. Απαριθμήστε όλες τις υποακολουθίες μήκους 2.
3. Είναι η $\langle B, D \rangle$ υποακολουθία της S ;
4. Είναι η $\langle C, B \rangle$ υποακολουθία της S ;

Λύση

(1) Αριθμός υποακολουθιών:

Κάθε στοιχείο μπορεί να είναι ή να μην είναι στην υποακολουθία:

$$\text{Total} = 2^n = 2^4 = 16 \text{ (συμπεριλαμβανομένης της κενής)}$$

(2) Υποακολουθίες μήκους 2:

$$\binom{4}{2} = 6 \text{ υποακολουθίες}$$

$\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle, \langle B, C \rangle, \langle B, D \rangle, \langle C, D \rangle$

(3) $\langle B, D \rangle$ υποακολουθία; **ΝΑΙ**

Το B εμφανίζεται πριν το D στην S : $A, \mathbf{B}, C, \mathbf{D}$

(4) $\langle C, B \rangle$ υποακολουθία; **ΟΧΙ**

Στην S το B έρχεται πριν το C, όχι μετά.

Υποακολουθία vs Υποσυμβολοσειρά

- **Subsequence:** Διατήρηση σειράς, όχι απαραίτητα συνεχόμενα
- **Substring:** Συνεχόμενα στοιχεία

12.7.2 Άσκηση 7.2: Forward Algorithm - Απλό**Εκφώνηση**

Ένα HMM έχει 2 καταστάσεις (H: Hot, C: Cold) και 2 παρατηρήσεις (S: Sun, R: Rain).

Παράμετροι:

- Initial: $\pi_H = 0.6, \pi_C = 0.4$
- Transitions: $a_{HH} = 0.7, a_{HC} = 0.3, a_{CH} = 0.4, a_{CC} = 0.6$
- Emissions: $b_H(S) = 0.8, b_H(R) = 0.2, b_C(S) = 0.3, b_C(R) = 0.7$

Υπολογίστε την πιθανότητα της παρατήρησης $O = \langle S, R \rangle$.

Λύση**Forward Algorithm:**

$$\alpha_t(j) = P(o_1, o_2, \dots, o_t, q_t = j | \lambda)$$

Βήμα 1: Αρχικοποίηση ($t = 1, o_1 = S$)

$$\alpha_1(H) = \pi_H \cdot b_H(S) = 0.6 \times 0.8 = 0.48$$

$$\alpha_1(C) = \pi_C \cdot b_C(S) = 0.4 \times 0.3 = 0.12$$

Βήμα 2: Induction ($t = 2, o_2 = R$)

$$\begin{aligned} \alpha_2(H) &= [\alpha_1(H) \cdot a_{HH} + \alpha_1(C) \cdot a_{CH}] \cdot b_H(R) \\ &= [0.48 \times 0.7 + 0.12 \times 0.4] \times 0.2 \\ &= [0.336 + 0.048] \times 0.2 = 0.384 \times 0.2 = 0.0768 \end{aligned}$$

$$\begin{aligned} \alpha_2(C) &= [\alpha_1(H) \cdot a_{HC} + \alpha_1(C) \cdot a_{CC}] \cdot b_C(R) \\ &= [0.48 \times 0.3 + 0.12 \times 0.6] \times 0.7 \\ &= [0.144 + 0.072] \times 0.7 = 0.216 \times 0.7 = 0.1512 \end{aligned}$$

Βήμα 3: Termination

$$P(O|\lambda) = \alpha_2(H) + \alpha_2(C) = 0.0768 + 0.1512 = 0.228$$

Τελική Απάντηση: $P(\langle S, R \rangle) = 0.228$

12.7.3 Άσκηση 7.3: Forward Algorithm - Πίνακας

Εκφώνηση

Με το ίδιο HMM, υπολογίστε $P(O = \langle S, S, R \rangle)$ χρησιμοποιώντας πίνακα.

Λύση

	$t = 1$ (S)	$t = 2$ (S)	$t = 3$ (R)
$\alpha_t(H)$	$0.6 \times 0.8 = 0.48$	0.2976	0.0655
$\alpha_t(C)$	$0.4 \times 0.3 = 0.12$	0.0648	0.1197

Υπολογισμοί $t = 2$:

$$\alpha_2(H) = [0.48 \times 0.7 + 0.12 \times 0.4] \times 0.8 = 0.384 \times 0.8 = 0.3072$$

$$\alpha_2(C) = [0.48 \times 0.3 + 0.12 \times 0.6] \times 0.3 = 0.216 \times 0.3 = 0.0648$$

Υπολογισμοί $t = 3$:

$$\alpha_3(H) = [0.3072 \times 0.7 + 0.0648 \times 0.4] \times 0.2 = 0.2409 \times 0.2 = 0.0482$$

$$\alpha_3(C) = [0.3072 \times 0.3 + 0.0648 \times 0.6] \times 0.7 = 0.1310 \times 0.7 = 0.0917$$

$$P(O) = 0.0482 + 0.0917 = 0.1399$$

12.7.4 Άσκηση 7.4: Viterbi Algorithm

Εκφώνηση

Με το ίδιο HMM, βρείτε την πιο πιθανή ακολουθία καταστάσεων για $O = \langle S, R \rangle$.

Λύση

Viterbi Algorithm:

$$\delta_t(j) = \max_{q_1, \dots, q_{t-1}} P(q_1, \dots, q_{t-1}, q_t = j, o_1, \dots, o_t | \lambda)$$

Βήμα 1: Αρχικοποίηση ($t = 1, o_1 = S$)

$$\delta_1(H) = \pi_H \cdot b_H(S) = 0.6 \times 0.8 = 0.48$$

$$\delta_1(C) = \pi_C \cdot b_C(S) = 0.4 \times 0.3 = 0.12$$

$$\psi_1(H) = \psi_1(C) = 0 \text{ (αρχή)}$$

Βήμα 2: Recursion ($t = 2, o_2 = R$)

$$\begin{aligned}
\delta_2(H) &= \max[\delta_1(H) \cdot a_{HH}, \delta_1(C) \cdot a_{CH}] \cdot b_H(R) \\
&= \max[0.48 \times 0.7, 0.12 \times 0.4] \times 0.2 \\
&= \max[0.336, 0.048] \times 0.2 = 0.336 \times 0.2 = 0.0672 \\
\psi_2(H) &= H \text{ (επειδή } 0.336 > 0.048)
\end{aligned}$$

$$\begin{aligned}
\delta_2(C) &= \max[\delta_1(H) \cdot a_{HC}, \delta_1(C) \cdot a_{CC}] \cdot b_C(R) \\
&= \max[0.48 \times 0.3, 0.12 \times 0.6] \times 0.7 \\
&= \max[0.144, 0.072] \times 0.7 = 0.144 \times 0.7 = 0.1008 \\
\psi_2(C) &= H \text{ (επειδή } 0.144 > 0.072)
\end{aligned}$$

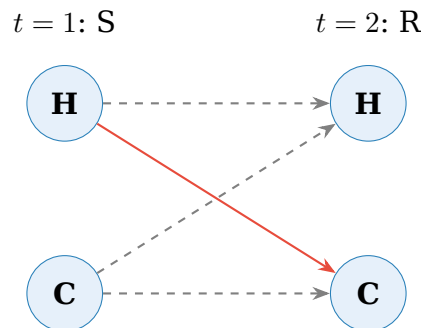
Βήμα 3: Termination

$$q_2^* = \arg \max[\delta_2(H), \delta_2(C)] = \arg \max[0.0672, 0.1008] = C$$

Βήμα 4: Backtracking

$$q_1^* = \psi_2(C) = H$$

Τελική Απάντηση: Πιο πιθανή ακολουθία: $\langle H, C \rangle$

**12.7.5 Άσκηση 7.5: Ερμηνεία HMM Παραμέτρων****Εκφώνηση**

Δίνεται HMM για αναγνώριση δραστηριότητας με καταστάσεις {Walk, Run, Sit} και παρατηρήσεις {Low, Medium, High} (accelerometer readings). Οι παράμετροι είναι:

A	Walk	Run	Sit	B	Low	Med	High
Walk	0.7	0.2	0.1	Walk	0.1	0.7	0.2
Run	0.1	0.8	0.1	Run	0.05	0.15	0.8
Sit	0.2	0.1	0.7	Sit	0.8	0.15	0.05

Τι μας λένε αυτές οι παράμετροι για τη συμπεριφορά του μοντέλου;

Λύση

Transition Matrix A - Ερμηνεία:

- Υψηλές διαγώνιες τιμές (0.7-0.8): Οι δραστηριότητες τείνουν να **συνεχίζονται**
- $a_{Walk \rightarrow Run} = 0.2$: Σχετικά πιθανή η μετάβαση από περπάτημα σε τρέξιμο
- $a_{Run \rightarrow Sit} = 0.1$: Σπάνια απότομη διακοπή τρεξίματος
- $a_{Sit \rightarrow Run} = 0.1$: Σπάνια άμεση μετάβαση από κάθισμα σε τρέξιμο

Emission Matrix B - Ερμηνεία:

- **Sit**: Κυρίως χαμηλές ενδείξεις (0.8) - λογικό, καθιστή θέση
- **Run**: Κυρίως υψηλές ενδείξεις (0.8) - έντονη κίνηση
- **Walk**: Κυρίως μεσαίες ενδείξεις (0.7) - μέτρια κίνηση

Χρησιμότητα HMM

Το HMM μπορεί να «εξομαλύνει» θορυβώδεις μετρήσεις χρησιμοποιώντας την πληροφορία ότι οι καταστάσεις τείνουν να παραμένουν σταθερές.

12.8 OLAP και Data Warehousing

12.8.1 Άσκηση 8.1: OLAP Operations Ταξινόμηση

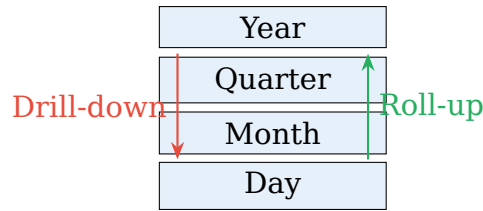
Εκφώνηση

Ταξινομήστε κάθε λειτουργία σε Roll-up, Drill-down, Slice, Dice, ή Pivot:

1. Από πωλήσεις ανά ημέρα σε πωλήσεις ανά μήνα
2. Εμφάνιση μόνο των πωλήσεων του 2024
3. Από πωλήσεις ανά χώρα σε πωλήσεις ανά πόλη
4. Πωλήσεις για συγκεκριμένη περιοχή ΚΑΙ κατηγορία προϊόντος
5. Αλλαγή από Product×Time σε Region×Time

Λύση

#	Operation	Εξήγηση
1	Roll-up	Aggregation σε υψηλότερο επίπεδο ιεραρχίας (ημέρα → μήνας)
2	Slice	Επιλογή μιας τιμής σε μία διάσταση (year = 2024)
3	Drill-down	Μετάβαση σε χαμηλότερο επίπεδο λεπτομέρειας
4	Dice	Επιλογή εύρους τιμών σε πολλές διαστάσεις
5	Pivot	Αναδιάταξη διαστάσεων/αλλαγή οπτικής γωνίας



12.8.2 Άσκηση 8.2: Roll-up και Drill-down Sequence

Εκφώνηση

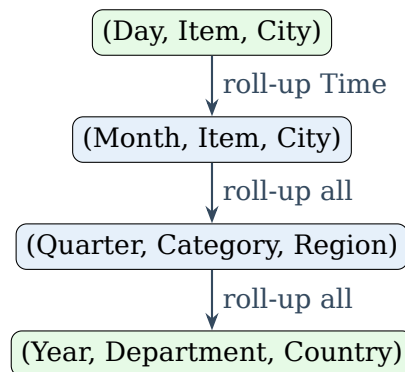
Δίνεται cube με διαστάσεις: Time (Day → Month → Quarter → Year), Product (Item → Category → Department), Location (City → Region → Country). Ξεκινώντας από (Day, Item, City), περιγράψτε τη σειρά operations για να φτάσετε σε (Year, Department, Country).

Λύση

Απαιτούμενα Roll-ups:

1. **Time:** Day → Month → Quarter → Year (3 roll-ups)
2. **Product:** Item → Category → Department (2 roll-ups)
3. **Location:** City → Region → Country (2 roll-ups)

Σύνολο: 7 consecutive roll-up operations



Optimization

Στην πράξη, τα roll-ups μπορούν να γίνουν παράλληλα σε διαφορετικές διαστάσεις αν υπάρχουν pre-computed aggregates.

12.8.3 Άσκηση 8.3: Slice vs Dice

Εκφώνηση

Για cube πωλήσεων με διαστάσεις (Product, Location, Time), εκφράστε σε OLAP operations:

1. Πωλήσεις για τον Ιανουάριο 2024

2. Πωλήσεις για Electronics σε Athens τον Ιανουάριο 2024
3. Πωλήσεις για όλα τα προϊόντα, στην Ελλάδα, για Q1 2024

Λύση

(1) Slice

$$\text{SLICE}(\text{Cube}, \text{Time} = \text{"Jan 2024"})$$

Μειώνει τον cube κατά μία διάσταση.

(2) Dice

$$\text{DICE}(\text{Cube}, \text{Product} = \text{"Electronics"}, \text{Location} = \text{"Athens"}, \text{Time} = \text{"Jan 2024"})$$

Επιλογή σε πολλαπλές διαστάσεις ταυτόχρονα.

(3) Slice + Roll-up (ή Dice)

1. Roll-up Location: City → Country (Ελλάδα)
2. Roll-up Time: Month → Quarter (Q1)
3. Slice: Country = "Greece", Quarter = "Q1 2024"

12.8.4 Άσκηση 8.4: Cube Calculations

Εκφώνηση

Ένας cube έχει 3 διαστάσεις: Product (100 values), Location (50 values), Time (365 days).

1. Πόσα cells έχει ο base cube;
2. Πόσα συνολικά cells έχει ο πλήρης CUBE (με όλα τα aggregations);
3. Αν κάθε cell χρειάζεται 8 bytes, πόση μνήμη απαιτείται;

Λύση

(1) Base Cube Cells:

$$|\text{Product}| \times |\text{Location}| \times |\text{Time}| = 100 \times 50 \times 365 = 1,825,000$$

(2) Full CUBE:

Για κάθε διάσταση, μπορεί να έχουμε τις τιμές της ή το ALL (aggregation):

$$\begin{aligned} & (|\text{Product}| + 1) \times (|\text{Location}| + 1) \times (|\text{Time}| + 1) \\ & = 101 \times 51 \times 366 = 1,884,666 \end{aligned}$$

Εναλλακτικά (για cuboids):

$$\text{Cuboids} = 2^3 = 8$$

Cuboid	Cells
(P, L, T)	$100 \times 50 \times 365 = 1,825,000$
(P, L, *)	$100 \times 50 = 5,000$
(P, *, T)	$100 \times 365 = 36,500$
(*, L, T)	$50 \times 365 = 18,250$
(P, *, *)	100
(*, L, *)	50
(*, *, T)	365
(*, *, *)	1
Total	1,885,266

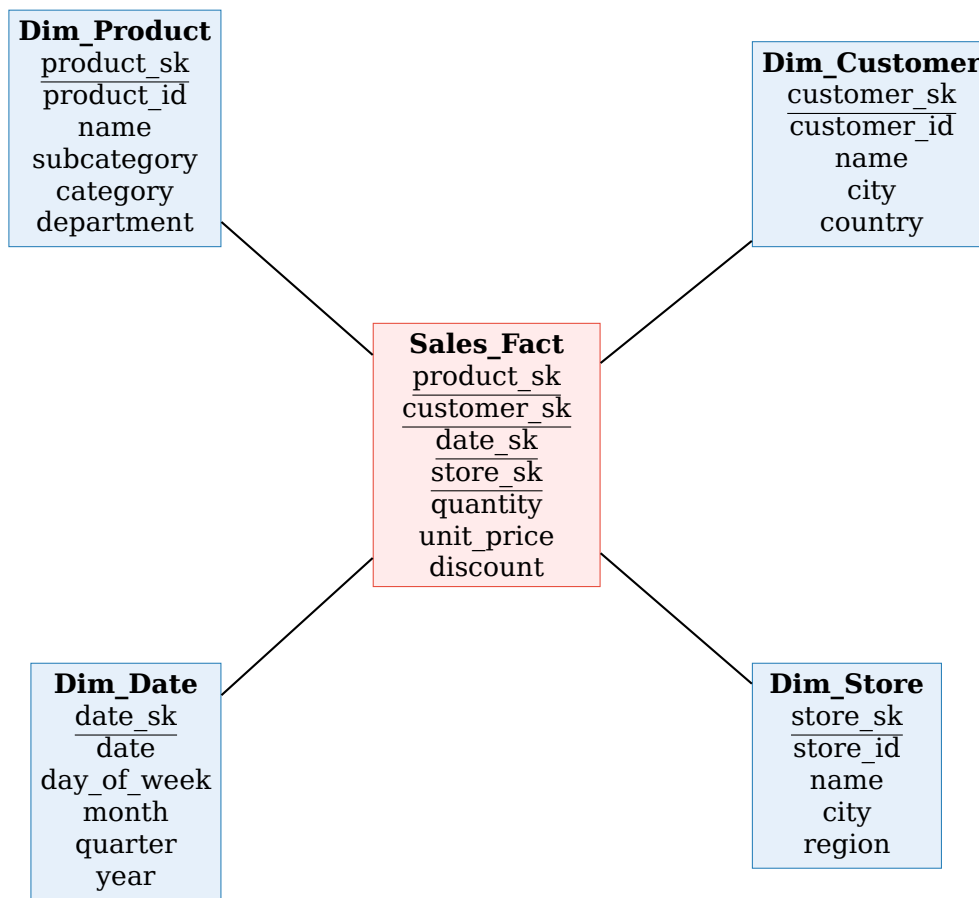
(3) Memory:

$$1,885,266 \times 8 \text{ bytes} = 15,082,128 \text{ bytes} \approx 14.4 \text{ MB}$$

12.8.5 Άσκηση 8.5: Star Schema Design**Εκφώνηση**

Σχεδιάστε star schema για ηλεκτρονικό κατάστημα με τα εξής requirements:

- Παρακολούθηση πωλήσεων (ποσότητα, τιμή, έκπτωση)
- Διαστάσεις: Προϊόν, Πελάτης, Ημερομηνία, Κατάστημα
- Ιεραρχία προϊόντων: Product → Subcategory → Category → Department

Λύση**Surrogate Keys (_sk)**

Χρησιμοποιούμε surrogate keys (τεχνητά κλειδιά) αντί για natural keys για:

- Καλύτερη απόδοση (μικρότερα integers)
- Διαχείριση slowly changing dimensions
- Ανεξαρτησία από source systems

12.8.6 Άσκηση 8.6: OLAP Queries σε SQL**Εκφώνηση**

Χρησιμοποιώντας το star schema της προηγούμενης άσκησης, γράψτε SQL queries για:

1. Συνολικές πωλήσεις ανά κατηγορία προϊόντος
2. Top 5 πελάτες με βάση το συνολικό ποσό αγορών
3. Πωλήσεις ανά μήνα για το 2024

Λύση**(1) Πωλήσεις ανά κατηγορία (Roll-up):**

```
1 SELECT
2     p.category,
3     SUM(f.quantity * f.unit_price * (1 - f.discount)) AS total_revenue
4 FROM Sales_Fact f
5 JOIN Dim_Product p ON f.product_sk = p.product_sk
6 GROUP BY p.category
7 ORDER BY total_revenue DESC;
```

(2) Top 5 πελάτες:

```
1 SELECT
2     c.name,
3     SUM(f.quantity * f.unit_price * (1 - f.discount)) AS total_spent
4 FROM Sales_Fact f
5 JOIN Dim_Customer c ON f.customer_sk = c.customer_sk
6 GROUP BY c.customer_sk, c.name
7 ORDER BY total_spent DESC
8 LIMIT 5;
```

(3) Πωλήσεις ανά μήνα 2024 (Slice + Roll-up):

```
1 SELECT
2     d.year,
3     d.month,
4     SUM(f.quantity * f.unit_price * (1 - f.discount)) AS monthly_revenue
5 FROM Sales_Fact f
6 JOIN Dim_Date d ON f.date_sk = d.date_sk
7 WHERE d.year = 2024
8 GROUP BY d.year, d.month
9 ORDER BY d.month;
```

Performance

Στα data warehouses, οι dimension tables είναι συνήθως denormalized για αποφυγή πολλαπλών JOINS. Αυτό αυξάνει το μέγεθος αλλά βελτιώνει την απόδοση των queries.

Σύνοψη Τύπων - Quick Reference

OLAP: Roll-up

Aggregation από λεπτομέρεια σε σύνοψη

Relational Algebra

$\sigma, \pi, \cup, \cap, -, \bowtie$

HMM: Viterbi

$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(o_t)$

Support

$Sup(X) = |T \text{ with } X|/|T|$

Confidence

$Conf(X \rightarrow Y) = Sup(X \cup Y)/Sup(X)$